Chapter 44

I2C Controller (I2C)

The I2C (Inter-Integrated Circuit) bus allows ESP32-P4 to communicate with multiple external devices. These external devices can share one I2C bus. ESP32-P4 has three I2C controllers: two in the main system and one in the low-power system. The two I2C controllers in the main system can act as a master or a slave (referred to as I2C below), while the one in the low-power system can only act as a master (referred to as LP_I2C below), which can still work when the main system sleeps.

44.1 Overview

The I2C bus has two lines, namely a serial data line (SDA) and a serial clock line (SCL). Both SDA and SCL lines are open-drain. The I2C bus can be connected to a single or multiple master devices and a single or multiple slave devices. However, only one master device can access a slave at a time via the bus.

The master initiates communication by generating a START condition: pulling the SDA line low while SCL is high. Then it issues nine clock pulses via SCL. The first eight pulses are used to transmit a 7-bit address followed by a read/write ($R/\overline{W}$) bit. If the address of an I2C slave matches the 7-bit address transmitted, this matching slave can respond by pulling SDA low on the ninth clock pulse. The master and the slave can send or receive data according to the $R/\overline{W}$ bit. Whether to terminate the data transfer or not is determined by the logic level of the acknowledge (ACK) bit. During data transfer, SDA changes only when SCL is low. Once the communication has finished, the master sends a STOP condition: pulling SDA up while SCL is high. If a master both reads and writes data in one transfer, then it should send a RSTART condition, a slave address, and a $R/\overline{W}$ bit before changing its operation. The RSTART condition is used to change the transfer direction and the mode of the devices (master mode or slave mode).

44.2 Features

The I2C controller of ESP32-P4 has the following features:

- Master mode and slave mode
- Communication between multiple masters and slaves
- Standard mode (100 Kbit/s)
- Fast mode (400 Kbit/s)
- 7-bit addressing and 10-bit addressing
- Continuous data transfer achieved by pulling SCL low in slave mode
- Programmable digital noise filtering
- Dual address mode, which uses slave address and slave memory or register address

44.3 I2C Architecture

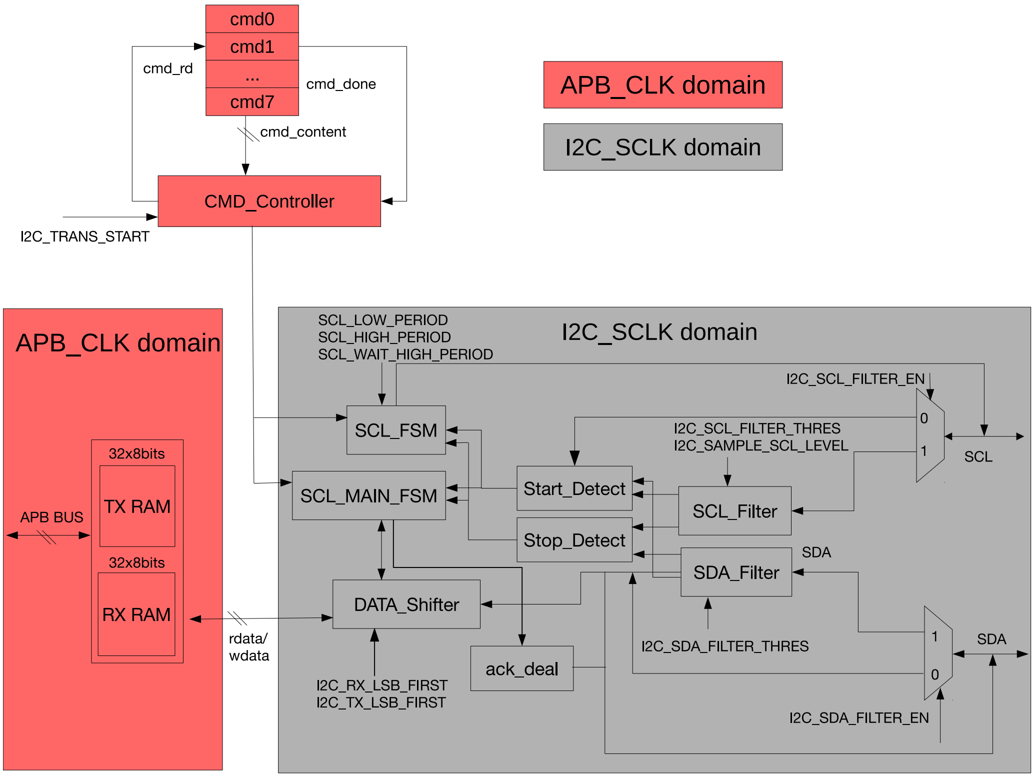

Figure 44.3-1. I2C Master Architecture

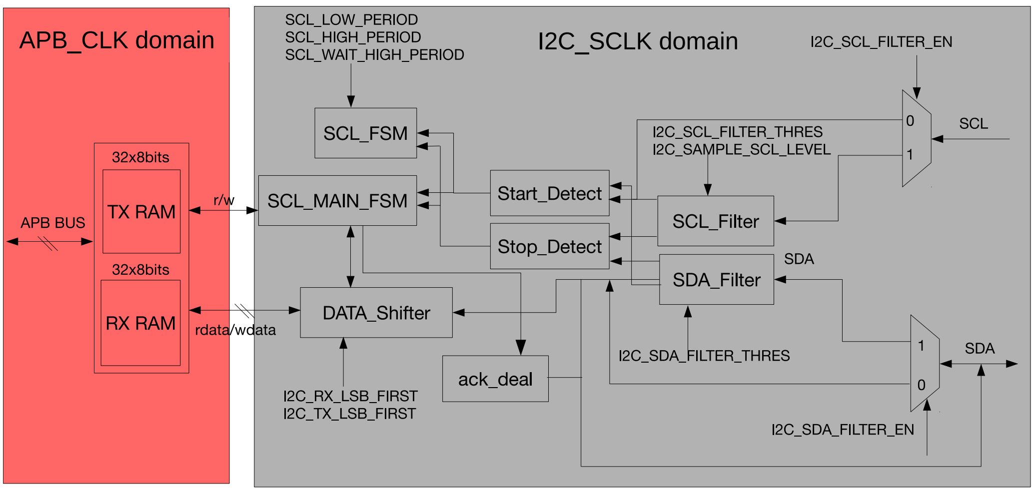

Figure 44.3-2. I2C Slave Architecture

The I2C controller runs either in master mode or slave mode, which is determined by I2C_MS_MODE. Figure 44.3-1 shows the architecture of a master, while Figure 44.3-2 shows that of a slave. The I2C controller has the following main parts:

- Transmit and receive memory (TX/RX RAM): store data to be transmitted and data received respectively.
- Command controller (CMD_Controller): generate (R)START, STOP, WRITE, READ, and END commands

- SCL clock controller (SCL_FSM): generate the timing sequence conforming to the I2C protocol. Figure 44.3-3 and Table 44.3-1 are the timing diagram and corresponding parameters of the I2C protocol.
- SDA data controller (SCL_MAIN_FSM): control the execution of I2C commands and the data sequence of the SDA line. It also controls the ACK_deal module to generate the ACK bit and detect the level of the ACK bit on the SDA line.
- Serial/parallel data converter (DATA_Shifter): shift data between serial and parallel form
- Filter for SCL (SCL_Filter): remove noises on SCL input signals
- Filter for SDA (SDA_Filter): remove noises on SDA input signals
- ACK bit controller (ack_deal): generate the ACK bit and detect the level of the ACK bit on the SDA line under the control of SCL_MAIN_FSM.

Besides, the I2C controller also has a clock module that generates I2C clocks, and a synchronization module that synchronizes the APB bus and the I2C controller.

The clock module is used to select clock sources, turn on and off clocks, and divide clocks. The synchronization module synchronizes signal transfer between different clock domains.

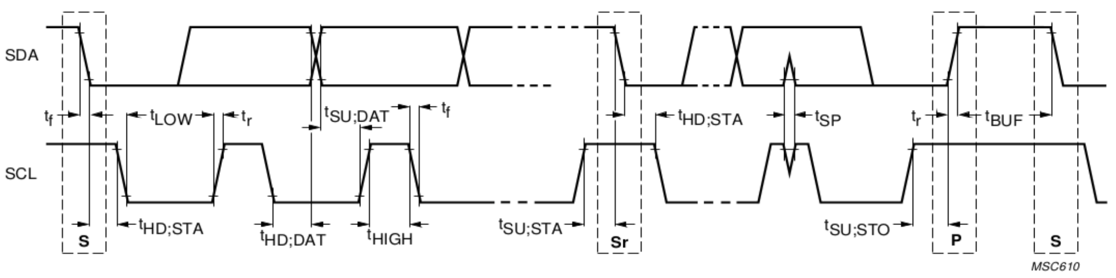

Fig.31 Definition of timing for F/S-mode devices on the I²C-bus.

Figure 44.3-3. I2C Protocol Timing (Cited from Fig.31 in The I2C-bus specification Version 2.1)

PARAMETER	SYMBOL	STANDARD-MODE		FAST-MODE		UNIT
		MIN.	MAX.	MIN.	MAX.	
SCL clock frequency	f_{SCL}	0	100	0	400	kHz
Hold time (repeated) START condition. After this period, the first clock pulse is generated	$t_{HD;STA}$	4.0	–	0.6	–	µs
LOW period of the SCL clock	t_{LOW}	4.7	–	1.3	–	µs
HIGH period of the SCL clock	t_{HIGH}	4.0	–	0.6	–	µs
Set-up time for a repeated START condition	$t_{SU;STA}$	4.7	–	0.6	–	µs
Data hold time: for CBUS compatible masters (see NOTE, Section 10.1.3) for I^2C-bus devices	$t_{HD;DAT}$	5.0 0(2)	– 3.45(3)	– 0(2)	– 0.9(3)	µs µs
Data set-up time	$t_{SU;DAT}$	250	–	100(4)	–	ns
Rise time of both SDA and SCL signals	t_r	–	1000	$20 + 0.1C_b$(5)	300	ns
Fall time of both SDA and SCL signals	t_f	–	300	$20 + 0.1C_b$(5)	300	ns
Set-up time for STOP condition	$t_{SU;STO}$	4.0	–	0.6	–	µs
Bus free time between a STOP and START condition	t_{BUF}	4.7	–	1.3	–	µs

Table 44.3-1. I2C Timing Parameters (Cited from Table 5 in The I2C-bus specification Version 2.1)

44.4 Functional Description

As mentioned above, one or more masters and one or more slaves can be connected on the I2C bus. The following sections describe the operations of the ESP32-P4 I2C controllers. Note that operations may differ between the I2C controllers in ESP32-P4 and other masters or slaves on the bus. Please refer to datasheets of individual I2C devices for specific information.

44.4.1 Clock Configuration

Registers, TX RAM, and RX RAM are configured and accessed in the APB_CLK clock domain. The main logic of the I2C controller, including SCL_FSM, SCL_MAIN_FSM, SCL_FILTER, SDA_FILTER, and DATA_SHIFTER, are in the I2C_SCLK clock domain.

You can configure the clock source for I2C_SCLK of I2C0 in the main system (HP) to FOSC_20M_CLK or XTAL_40M_CLK via HP_SYS_CLKRST_I2C0_CLK_SRC_SEL and configure that of I2C1 in the main system via HP_SYS_CLKRST_I2C1_CLK_SRC_SEL. For the clock source for I2C_SCLK of LP_I2C in the low-power system (LP), you can configure it to XTAL_D2_CLK, LP_DYN_FAST_CLK, or PLL_LP_CLK 8 MHz via LPPERI_LP_I2C_CLK_SEL.

The steps to configure the clock source for HP I2C0 are as follows:

- Enable the clock source for I2C_SCLK of I2C0 by configuring HP_SYS_CLKRST_I2C0_CLK_EN to 1.
- When HP_SYS_CLKRST_I2C0_SRC_SEL is 0, the clock source is XTAL_40M_CLK.
- When HP_SYS_CLKRST_I2C0_SRC_SEL is 1, the clock source is FOSC_20M_CLK.

The steps to configure the clock source for HP I2C1 are as follows:

- Enable the clock source for I2C_SCLK of I2C1 by configuring HP_SYS_CLKRST_I2C1_CLK_EN to 1.

- When HP_SYS_CLKRST_I2C1_SRC_SEL is 0, the clock source is XTAL_40M_CLK.
- When HP_SYS_CLKRST_I2C1_SRC_SEL is 1, the clock source is FOSC_20M_CLK.

The steps to configure the clock source for LP_I2C are as follows:

- Enable the clock source for I2C_SCLK of LP_I2C by configuring LPPERI_CK_EN_LP_I2C to 1.
- When LPPERI_LP_I2C_CLK_SEL is 0, the clock source is LP_DYN_FAST_CLK.
- When LPPERI_LP_I2C_CLK_SEL is 1, the clock source is XTAL_D2_CLK.
- When LPPERI_LP_I2C_CLK_SEL is 2, the clock source is PLL_LP_CLK 8 MHz.

The clock source then passes through a fractional divider to generate I2C_SCLK according to the following equation:

$$I2C\_SCLK\_DIV\_NUM + 1 + \frac{I2C\_SCLK\_DIV\_A}{I2C\_SCLK\_DIV\_B}$$

In the equation, I2C_SCLK_DIV_NUM represents the integer part of the divisor, I2C_SCLK_DIV_A represents the numerator of the fractional part of the divisor, and I2C_SCLK_DIV_B represents the denominator of the fractional part of the divisor. Limited by timing parameters, the derived clock I2C_SCLK should operate at a frequency 20 times larger than SCL's frequency.
For I2C0:

- Configure I2C_SCLK_DIV_NUM via HP_SYS_CLKRST_I2C0_CLK_DIV_NUM.
- Configure I2C_SCLK_DIV_A via HP_SYS_CLKRST_I2C0_CLK_DIV_NUMERATOR.
- Configure I2C_SCLK_DIV_B via HP_SYS_CLKRST_I2C0_CLK_DIV_DENOMINATOR.

For I2C1:

- Configure I2C_SCLK_DIV_NUM via HP_SYS_CLKRST_I2C1_CLK_DIV_NUM.
- Configure I2C_SCLK_DIV_A via HP_SYS_CLKRST_I2C1_CLK_DIV_NUMERATOR.
- Configure I2C_SCLK_DIV_B via HP_SYS_CLKRST_I2C1_CLK_DIV_DENOMINATOR.

44.4.2 SCL and SDA Noise Filtering

SCL_Filter and SDA_Filter modules are identical and are used to filter signal noise on SCL and SDA, respectively. These filters can be enabled or disabled by configuring I2C_SCL_FILTER_EN and I2C_SDA_FILTER_EN.

Take SCL_Filter as an example. When enabled, SCL_Filter samples input signals on the SCL line continuously. These input signals are valid only if they remain unchanged for consecutive I2C_SCL_FILTER_THRES I2C_SCLK clock cycles. Given that only valid input signals can pass through the filter, SCL_Filter can remove glitches whose pulse width is shorter than I2C_SCL_FILTER_THRES I2C_SCLK clock cycles, while SDA_Filter can remove glitches whose pulse width is shorter than I2C_SDA_FILTER_THRES I2C_SCLK clock cycles.

44.4.3 SCL Clock Stretching

The I2C controller in slave mode (i.e., slave) can realize the function called clock stretching by holding the SCL line low to suspend data transmission in exchange for more time to process data. This function is enabled by setting the I2C_SLAVE_SCL_STRETCH_EN bit. The time period to release the SCL line from stretching is configured by setting the I2C_STRETCH_PROTECT_NUM field, in order to avoid timing sequence errors. The slave can choose to achieve clock stretching by holding the SCL line low when one of the following four events occurs:

1. Address match: The address of the slave matches the address sent by the master via the SDA line, and the $R/\overline{W}$ bit is 1.
2. RAM being full: RX RAM of the slave is full. Note that when the slave receives less than the FIFO depth, which is 32 bytes in ESP32-P4 I2C, it is not necessary to enable clock stretching; when the slave receives FIFO depth bytes or more, you may interrupt data transmission to wrapped around RAM via the FIFO threshold, or enable clock stretching for more time to process data. When clock stretching is enabled, I2C_RX_FULL_ACK_LEVEL must be cleared, otherwise there will be unpredictable consequences.
3. RAM being empty: The slave is sending data, but its TX RAM is empty.
4. Sending an ACK: If I2C_SLAVE_BYTE_ACK_CTL_EN is set, the slave pulls SCL low when sending an ACK bit. At this stage, software validates data and configures I2C_SLAVE_BYTE_ACK_LVL to control the level of the ACK bit. Note that when RX RAM of the slave is full, the level of the ACK bit to be sent is determined by I2C_RX_FULL_ACK_LEVEL, instead of I2C_SLAVE_BYTE_ACK_LVL. In this case, I2C_RX_FULL_ACK_LEVEL should also be cleared to ensure proper functioning of clock stretching.

When clock stretching occurs, the cause of stretching can be read from the I2C_STRETCH_CAUSE bit. Clock stretching can be disabled by setting the I2C_SLAVE_SCL_STRETCH_CLR bit.

44.4.4 Generating SCL Pulses in Idle State

Usually, when the I2C bus is idle, the SCL line is held high. The I2C controller in ESP32-P4 can be programmed to generate SCL pulses in an idle state. This function only works when the I2C controller is configured as master. If the I2C_SCL_RST_SLV_EN bit is set, hardware will send I2C_SCL_RST_SLV_NUM SCL pulses, and then automatically clear the I2C_SCL_RST_SLV_EN bit.

44.4.5 Synchronization

I2C registers are configured in the APB_CLK domain, whereas the I2C controller is configured in the asynchronous I2C_SCLK domain. Therefore, before being used by the I2C controller, register values should be synchronized by first writing configuration registers and then writing 1 to I2C_CONF_UPGATE. Registers that need synchronization are listed in Table 44.4-1.

Table 44.4-1. I2C Synchronous Registers

Register	Field	Address
I2C_CTR_REG	I2C_SLV_TX_AUTO_START_EN	0x0004
	I2C_ADDR_10BIT_RW_CHECK_EN	
	I2C_ADDR_BROADCASTING_EN	
	I2C_SDA_FORCE_OUT	
	I2C_SCL_FORCE_OUT	
	I2C_SAMPLE_SCL_LEVEL	
	I2C_RX_FULL_ACK_LEVEL	
	I2C_MS_MODE	
	I2C_TX_LSB_FIRST	
	I2C_RX_LSB_FIRST	
	I2C_ARBITRATION_EN	
I2C_TO_REG	I2C_TIME_OUT_EN	0x000C
	I2C_TIME_OUT_VALUE	
I2C_SLAVE_ADDR_REG	I2C_ADDR_10BIT_EN	0x0010
	I2C_SLAVE_ADDR	
I2C_FIFO_CONF_REG	I2C_FIFO_ADDR_CFG_EN	0x0018
I2C_SCL_SP_CONF_REG	I2C_SDA_PD_EN	0x0080
	I2C_SCL_PD_EN	
	I2C_SCL_RST_SLV_NUM	
	I2C_SCL_RST_SLV_EN	
I2C_SCL_STRETCH_CONF_REG	I2C_SLAVE_BYTE_ACK_CTL_EN	0x0084
	I2C_SLAVE_BYTE_ACK_LVL	
	I2C_SLAVE_SCL_STRETCH_EN	
	I2C_STRETCH_PROTECT_NUM	
I2C_SCL_LOW_PERIOD_REG	I2C_SCL_LOW_PERIOD	0x0000
I2C_SCL_HIGH_PERIOD_REG	I2C_WAIT_HIGH_PERIOD	0x0038
	I2C_HIGH_PERIOD	
I2C_SDA_HOLD_REG	I2C_SDA_HOLD_TIME	0x0030
I2C_SDA_SAMPLE_REG	I2C_SDA_SAMPLE_TIME	0x0034
I2C_SCL_START_HOLD_REG	I2C_SCL_START_HOLD_TIME	0x0040
I2C_SCL_RSTART_SETUP_REG	I2C_SCL_RSTART_SETUP_TIME	0x0044
I2C_SCL_STOP_HOLD_REG	I2C_SCL_STOP_HOLD_TIME	0x0048
I2C_SCL_STOP_SETUP_REG	I2C_SCL_STOP_SETUP_TIME	0x004C
I2C_SCL_ST_TIME_OUT_REG	I2C_SCL_ST_TO_I2C	0x0078
I2C_SCL_MAIN_ST_TIME_OUT_REG	I2C_SCL_MAIN_ST_TO_I2C	0x007C
I2C_FILTER_CFG_REG	I2C_SCL_FILTER_EN	0x0050
	I2C_SCL_FILTER_THRES	
	I2C_SDA_FILTER_EN	
	I2C_SDA_FILTER_THRES	

44.4.6 Open-Drain Output

SCL and SDA output drivers must be configured as open-drain. There are two ways to achieve this:

1. Set I2C_SCL_FORCE_OUT and I2C_SDA_FORCE_OUT, and configure GPIO_PINn_PAD_DRIVER for corresponding SCL and SDA pads as open-drain.
2. Clear I2C_SCL_FORCE_OUT and I2C_SDA_FORCE_OUT.

Because these lines are configured as open-drain, the low-to-high transition time of each line is longer, determined together by the pull-up resistor and line capacitance. The output duty cycle of I2C is limited by the SDA and SCL line's pull-up speed, mainly SCL's speed.

In addition, when I2C_SCL_FORCE_OUT and I2C_SCL_PD_EN are set to 1, SCL can be forced low; when I2C_SDA_FORCE_OUT and I2C_SDA_PD_EN are set to 1, SDA can be forced low.

44.4.7 Timing Parameter Configuration

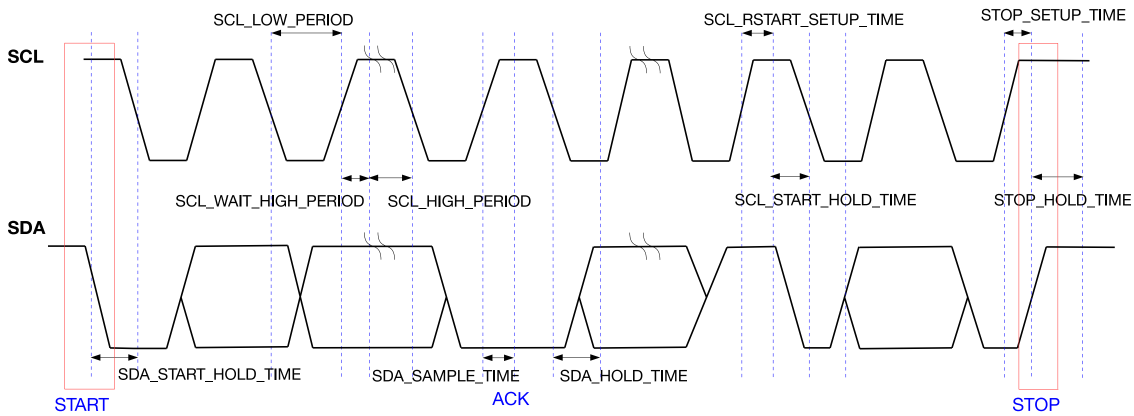

Figure 44.4-1. I2C Timing Diagram

Figure 44.4-1 shows the timing diagram of an I2C master. This figure also specifies registers used to configure the START bit, STOP bit, data hold time, data sample time, waiting time on the rising SCL edge, etc. Timing parameters are calculated as follows in I2C_SCLK clock cycles:

1. $t_{LOW} = (I2C\_SCL\_LOW\_PERIOD + 1) \cdot T_{I2C\_SCLK}$
2. $t_{HIGH} = (I2C\_SCL\_HIGH\_PERIOD + 1) \cdot T_{I2C\_SCLK}$
3. $t_{SU:STA} = (I2C\_SCL\_RSTART\_SETUP\_TIME + 1) \cdot T_{I2C\_SCLK}$
4. $t_{HD:STA} = (I2C\_SCL\_START\_HOLD\_TIME + 1) \cdot T_{I2C\_SCLK}$
5. $t_r = (I2C\_SCL\_WAIT\_HIGH\_PERIOD + 1) \cdot T_{I2C\_SCLK}$
6. $t_{SU:STO} = (I2C\_SCL\_STOP\_SETUP\_TIME + 1) \cdot T_{I2C\_SCLK}$
7. $t_{BUF} = (I2C\_SCL\_STOP\_HOLD\_TIME + 1) \cdot T_{I2C\_SCLK}$
8. $t_{HD:DAT} = (I2C\_SDA\_HOLD\_TIME + 1) \cdot T_{I2C\_SCLK}$
9. $t_{SU:DAT} = (I2C\_SCL\_LOW\_PERIOD - I2C\_SDA\_HOLD\_TIME) \cdot T_{I2C\_SCLK}$

Timing registers below are divided into two groups, depending on the mode in which these registers are active:

- Master mode only:

1. I2C_SCL_START_HOLD_TIME: Specifies the interval between the moment SDA is pulled low and the moment SCL is pulled low when the master generates a START condition. This interval is (I2C_SCL_START_HOLD_TIME +1) in I2C_SCLK cycles. This register is active only when the I2C controller works in master mode.
2. I2C_SCL_LOW_PERIOD: Specifies the low period of SCL. This period lasts (I2C_SCL_LOW_PERIOD +1) in I2C_SCLK cycles. This register is active only when the I2C controller works in master mode.

   However, this period could be extended in the following scenarios:

   - SCL is pulled low by peripheral devices when I2C acts as a master.
   - SCL is pulled low by an END command executed by the I2C controller.
   - SCL is pulled low by clock stretching when I2C acts as a slave.
3. I2C_SCL_WAIT_HIGH_PERIOD: Specifies the time for SCL to switch from low to high in I2C_SCLK cycles. Please make sure that SCL can be pulled high within this time period. Otherwise, the high period of SCL may be incorrect. This register is active only when the I2C controller works in master mode.
4. I2C_SCL_HIGH_PERIOD: Specifies the high period of SCL in I2C_SCLK cycles. This register is active only when the I2C controller works in master mode. When SCL goes high within (I2C_SCL_WAIT_HIGH_PERIOD + 1) in I2C_SCLK cycles, its frequency is:
   $$f_{scl} = \frac{f_{\text{I2C\_SCLK}}}{\text{I2C\_SCL\_LOW\_PERIOD} + \text{I2C\_SCL\_HIGH\_PERIOD} + \text{I2C\_SCL\_WAIT\_HIGH\_PERIOD} + 3 + \text{I2C\_SCL\_FILTER\_THRES}}$$

   where 3 represents the amount of clock cycles required to synchronize the SCL. If the SCL filtering function is turned on, the delay caused by I2C_SCL_FILTER_THRES needs to be added. As the SCL low-to-high transition time represented by I2C_SCL_WAIT_HIGH_PERIOD + 1 module clock can be affected by the pull-up resistor, IO drive capability, SCL line capacitance, etc., deviation may occur between the actual frequency of the test and the theoretical frequency. At this point, deviations can be reduced by adjusting the value of I2C_SCL_WAIT_HIGH_PERIOD.

- Master mode and slave mode:

  1. I2C_SDA_SAMPLE_TIME: Specifies the interval between the rising edge of SCL and the level sampling time of SDA. It is advised to set a value in the middle of SCL's high period, so as to correctly sample the level of SCL. This register is active both in master mode and slave mode.
  2. I2C_SDA_HOLD_TIME: Specifies the interval between changing the SDA output level and the falling edge of SCL. This register is active both in master mode and slave mode.

Timing parameters limits corresponding register configuration.

1. $\frac{f_{I2C\_SCLK}}{f_{SCL}} > 20$
2. $3 \times f_{I2C\_SCLK} \leq (I2C\_SDA\_HOLD\_TIME - 4) \times f_{APB\_CLK}$
3. I2C_SDA_HOLD_TIME + I2C_SCL_START_HOLD_TIME > SDA_FILTER_THRES + 3
4. I2C_SCL_WAIT_HIGH_PERIOD < I2C_SDA_SAMPLE_TIME < I2C_SCL_HIGH_PERIOD
5. I2C_SDA_SAMPLE_TIME < I2C_SCL_WAIT_HIGH_PERIOD + I2C_SCL_START_HOLD_TIME + I2C_SCL_RSTART_SETUP_TIME
6. I2C_STRETCH_PROTECT_NUM + I2C_SDA_HOLD_TIME > I2C_SCL_LOW_PERIOD

44.4.8 Timeout Control

The I2C controller has three types of timeout control, namely timeout control for SCL_FSM, for SCL_MAIN_FSM, and for the SCL line. The first two are always enabled, while the third is configurable.

When SCL_FSM remains unchanged for more than $2^{I2C\_SCL\_ST\_TO\_I2C+1}$ –1 clock cycles, an I2C_SCL_ST_TO_INT interrupt is triggered, and then SCL_FSM goes to idle state. The value of I2C_SCL_ST_TO_I2C should be less than or equal to 23, which means SCL_FSM could remain unchanged for 2^{24} –1 I2C_SCLK clock cycles at most before the interrupt is generated.

When SCL_MAIN_FSM remains unchanged for more than $2^{I2C\_SCL\_MAIN\_ST\_TO\_I2C+1}$ –1 I2C_SCLK clock cycles, an I2C_SCL_MAIN_ST_TO_INT interrupt is triggered, and then SCL_MAIN_FSM goes to idle state. The value of I2C_SCL_MAIN_ST_TO_I2C should be less than or equal to 23, which means SCL_MAIN_FSM could remain unchanged for 2^{24} –1 clock cycles at most before the interrupt is generated.

Timeout control for SCL is enabled by setting I2C_TIME_OUT_EN. When the level of SCL remains unchanged for more than $2^{I2C\_TIME\_OUT\_VALUE+1}$ –1 clock cycles, an I2C_TIME_OUT_INT interrupt is triggered, and then the I2C bus goes to idle state.

44.4.9 Command Configuration

When the I2C controller works in master mode, CMD_Controller reads commands from 8 sequential command registers and controls SCL_FSM and SCL_MAIN_FSM accordingly.

	31	30:14	13:11	10	9	8	7:0
cmd0	CMD_DONE	reserved	op_code	ack_value	ack_exp	ack_check_en	byte_num

	31	30:14	13:11	10	9	8	7:0
cmd7	CMD_DONE	reserved	op_code	ack_value	ack_exp	ack_check_en	byte_num

Figure 44.4-2. Structure of I2C Command Registers

Command registers, whose structure is illustrated in Figure 44.4-2, are active only when the I2C controller works in master mode. Fields of command registers are:

1. CMD_DONE: Indicates that a command has been executed. After each command has been executed, the CMD_DONE bit in the corresponding command register is set to 1 by hardware. By reading this bit, the software can tell if the command has been executed. When writing new commands, this bit must be cleared by software.

2. op_code: Indicates the command. The I2C controller supports five commands:

   - WRITE: op_code = 1. The I2C controller sends a slave address, a register address (only in dual address mode), and data to the slave.

   - STOP: op_code = 2. The I2C controller sends a STOP bit defined by the I2C protocol. This code also indicates that the command sequence has been executed, and the CMD_Controller stops reading commands. After restarted by software, the CMD_Controller resumes reading commands from command register 0.

- READ: op_code = 3. The I2C controller reads data from the slave.
- END: op_code = 4. The I2C controller pulls the SCL line down and suspends I2C communication. This code also indicates that the command sequence has been completed, and the CMD_Controller stops executing commands. Once the software refreshes data in command registers and the RAM, the CMD_Controller can be restarted to execute commands from command register 0 again.
- RSTART: op_code = 6. The I2C controller sends a START bit or a RSTART bit defined by the I2C protocol.

3. ack_value: Used to configure the level of the ACK bit sent by the I2C controller during a read operation. This bit is ignored in RSTART, STOP, END, and WRITE conditions.
4. ack_exp: Used to configure the level of the ACK bit expected by the I2C controller during a write operation. This bit is ignored during RSTART, STOP, END, and READ conditions.
5. ack_check_en: Used to enable the I2C controller during a write operation to check whether the ACK level sent by the slave matches ack_exp in the command. If this bit is set and the level received does not match ack_exp in the WRITE command, the master will generate an I2C_NACK_INT interrupt and a STOP condition for data transfer. If this bit is cleared, the controller will not check the ACK level sent by the slave. This bit is ignored during RSTART, STOP, END, and READ conditions.
6. byte_num: Specifies the length of data (in bytes) to be read or written. It can range from 1 to 255 bytes. This bit is ignored during RSTART, STOP, and END conditions.

Each command sequence is executed starting from command register 0 and terminated by a STOP or an END. Therefore, there must be a STOP or an END command in the eight command registers.

A complete data transfer on the I2C bus should be initiated by a START and terminated by a STOP. The transfer process may be completed using multiple sequences, separated by END commands. Each sequence may differ in the direction of data transfer, clock frequency, slave addresses, data length, etc. This allows efficient use of available peripheral RAM and also achieves more flexible I2C communication.

44.4.10 TX/RX RAM Data Storage

Both TX RAM and RX RAM are 32 × 8 bits and can be accessed in FIFO or non-FIFO mode. If I2C_NONFIFO_EN bit is cleared, both RAMs are accessed in FIFO mode; if I2C_NONFIFO_EN bit is set, both RAMs are accessed in non-FIFO mode.

TX RAM stores data that the I2C controller needs to send. During communication, when the I2C controller needs to send data (except acknowledgment bits), it reads data from TX RAM and sends them sequentially via SDA. When the I2C controller works in master mode, all data must be stored in TX RAM in the order they need to be sent to slaves. The data stored in TX RAM include slave addresses, read/write bits, register addresses (only in dual address mode) and data to be sent. When the I2C controller works in slave mode, TX RAM only stores data to be sent.

TX RAM can be read and written by the CPU. The CPU writes to TX RAM either in FIFO mode or in non-FIFO mode (direct address). In FIFO mode, the CPU writes to TX RAM via the fixed address I2C_DATA_REG, with addresses for writing in TX RAM incremented automatically by hardware. In non-FIFO mode, the CPU accesses TX RAM directly via address fields (I2C Base Address + 0x100) ~(I2C Base Address + 0x17C). Each byte in TX RAM occupies an entire word in the address space. Therefore, the address of the first byte is I2C Base

Address + 0x100, the second byte is I2C Base Address + 0x104, the third byte is I2C Base Address + 0x108, and so on. The CPU can only read TX RAM via direct addresses. Bytes written to the TX RAM can be read back by the CPU, via the direct addresses. Addresses for reading TX RAM are the same as addresses for writing TX RAM.

RX RAM stores data the I2C controller receives during communication. When the I2C controller works in slave mode, neither slave addresses sent by the master nor register addresses (only in dual address mode) will be stored in RX RAM. Values of RX RAM can be read by software after I2C communication is completed.

RX RAM can only be read by the CPU. The CPU reads RX RAM either in FIFO mode or in non-FIFO mode (direct address). In FIFO mode, the CPU reads RX RAM via the fixed address I2C_DATA_REG, with addresses for reading RX RAM incremented automatically by hardware. In non-FIFO mode, the CPU accesses RX RAM directly via address fields (I2C Base Address + 0x180) ~(I2C Base Address + 0x1FC). Each byte in RX RAM occupies an entire word in the address space. Therefore, the address of the first byte is I2C Base Address + 0x180, the second byte is I2C Base Address + 0x184, the third byte is I2C Base Address + 0x188 and so on.

In FIFO mode, the TX RAM of a master may wrap around to send data larger than the FIFO depth. Set I2C_FIFO_PRT_EN. If the size of data to be sent is smaller than I2C_TXFIFO_WM_THRHD (master), an I2C_TXFIFO_WM_INT (master) interrupt is generated. After receiving the interrupt, the software continues writing to I2C_DATA_REG (master). Please ensure that software writes to or refreshes TX RAM before the master sends data, otherwise it may result in unpredictable consequences.

In FIFO mode, the RX RAM of a slave may also wrap around to receive data larger than the FIFO depth. Set I2C_FIFO_PRT_EN and clear I2C_RX_FULL_ACK_LEVEL. If data already received (to be overwritten) is larger than I2C_RXFIFO_WM_THRHD (slave), an I2C_RXFIFO_WM_INT (slave) interrupt is generated. After receiving the interrupt, the software continues reading from I2C_DATA_REG (slave).

44.4.11 Data Conversion

DATA_Shifter is used for serial/parallel conversion, converting byte data in TX RAM to an outgoing serial bitstream or an incoming serial bitstream to byte data in RX RAM. I2C_RX_LSB_FIRST and I2C_TX_LSB_FIRST can be used to select LSB- or MSB-first storage and transmission of data.

44.4.12 Addressing Mode

The ESP32-P4 I2C controller supports 7-bit and 10-bit addressing. 10-bit addressing can be mixed with 7-bit addressing. Besides, the ESP32-P4 I2C controller also supports dual address mode.

Define the slave address as SLV_ADDR. In 7-bit addressing mode, the slave address is SLV_ADDR[6:0]; in 10-bit addressing mode, the slave address is SLV_ADDR[9:0].

In 7-bit addressing mode, the master only needs to send one byte of the address, which comprises SLV_ADDR[6:0] and a $R/\overline{W}$ bit. In the 7-bit addressing mode, there is a special case called general call addressing (broadcast). It is enabled by setting I2C_ADDR_BROADCASTING_EN in a slave. When the slave receives the general call address (0x00) from the master and the $R/\overline{W}$ bit followed is 0, it responds to the master regardless of its own address.

In 10-bit addressing mode, the master needs to send two bytes of address. The first byte is slave_addr_first_7bits followed by a $R/\overline{W}$ bit, and slave_addr_first_7bits should be configured as (0x78 | SLV_ADDR[9:8]). The second byte is slave_addr_second_byte, which should be configured as SLV_ADDR[7:0].

The slave can enable 10-bit addressing by configuring I2C_ADDR_10BIT_EN. I2C_SLAVE_ADDR is used to configure I2C slave address. Specifically, I2C_SLAVE_ADDR[14:7] should be configured as SLV_ADDR[7:0], and I2C_SLAVE_ADDR[6:0] should be configured as (0x78 | SLV_ADDR[9:8]). Since a 10-bit slave address has one more byte than a 7-bit address, byte_num of the WRITE command and the number of bytes in the RAM increase by one. Please refer to Programming Example for detailed descriptions.

When working in slave mode, the I2C controller supports dual address mode, where the first address is the address of an I2C slave, and the second one is the slave's memory address. When using dual address mode, RAM must be accessed in non-FIFO mode. Dual address mode is enabled by setting I2C_FIFO_ADDR_CFG_EN. When the slave address received by the slave is inconsistent with the internally configured slave address, the I2C_SLAVE_ADDR_UNMATCH interrupt will be generated.

44.4.13 $R/\overline{W}$ Bit Check in 10-bit Addressing Mode

In 10-bit addressing mode, when I2C_ADDR_10BIT_RW_CHECK_EN is set to 1, the I2C controller performs a check on the first byte, which consists of slave_addr_first_7bits and a $R/\overline{W}$ bit. When the $R/\overline{W}$ bit does not indicate a WRITE operation, i.e., not in line with the I2C protocol, the data transfer ends. If the check feature is not enabled, when the $R/\overline{W}$ bit does not indicate a WRITE, the data transfer still continues, but transfer failure may occur.

44.4.14 To Start the I2C Controller

To start the I2C controller in master mode, after configuring the controller to master mode and command registers, write 1 to I2C_TRANS_START in order to let the master start to parse and execute command sequences. The master always executes a command sequence starting from the command register 0 to a STOP or an END. To execute another command sequence starting from command register 0, refresh commands by writing 1 again to I2C_TRANS_START.

There are two ways to start the I2C controller in slave mode:

- **Automatic Start Transmission:** When the register I2C_SLV_TX_AUTO_START_EN is set, the slave automatically starts transmission after being addressed by the master. This mode is suitable when the master needs to continuously access slave data.
- **Manual Start Transmission:** When the register I2C_SLV_TX_AUTO_START_EN is cleared, the slave must explicitly set I2C_TRANS_START before each transmission. This mode is suitable when the master needs to selectively read slave data.

  For example:

  1. The slave prepares 5 bytes of data;
  2. The master reads only the first 3 bytes;
  3. After the master finishes reading, the slave sets I2C_TX_FIFO_RST to clear the remaining data in TXFIFO, and then sets I2C_TRANS_START to reinitialize the slave state.

Notes: In manual start mode, I2C_TRANS_START must be set before the next address match. If the master addresses the slave while I2C_TRANS_START is not set, the slave may acknowledge the address but fail to progress its internal state machines, which can lead to undefined behavior or timing disorder in the slave controller.

44.5 Functional Differences Between LP_I2C and I2C

LP_I2C can be used as a master to communicate with external devices when the main system sleeps. LP_I2C includes all the functions of the ESP32-P4 $I2C_{master}$, but doesn't include any functions of ESP32-P4 $I2C_{slave}$. It does not contain any registers related to the $I2C_{slave}$. For detailed register list, see 44.8.2 LP_I2C Register Summary.

The design differences between LP_I2C and I2C master are as follows:

- The size of TX/RX RAM in LP_I2C is 16*8 bit, which means the TX/RX FIFO depth is 16 bytes.
- The clock source of APB_CLK in LP_I2C is CLK_AON_FAST. The clock source for I2C_SCLK is configured via LPPERI_LP_I2C_CLK_SEL. The corresponding bits are:
  - 0: CLK_ROOT_FAST
  - 1: CLK_XTALD2
  - 2: PLL_F8M_CLK

  Configuring LPPERI_CK_EN_LP_I2C to 1 enables the clock source of I2C_SCLK. Adjust the timing registers accordingly when the clock frequency changes.

See the programming examples of ESP32-P4 $I2C_{slave}$ in 44.6 for that of LP_I2C.

44.6 Programming Example

This section provides programming examples for typical communication scenarios. ESP32-P4 has two I2C controllers. For the convenience of description, I2C masters and slaves in all subsequent figures are ESP32-P4 I2C controllers. I2C master is referred to as $I2C_{master}$, and I2C slave is referred to as $I2C_{slave}$.

44.6.1 $I2C_{master}$ Writes to $I2C_{slave}$ with a 7-bit Address in One Command Sequence

44.6.1.1 Introduction

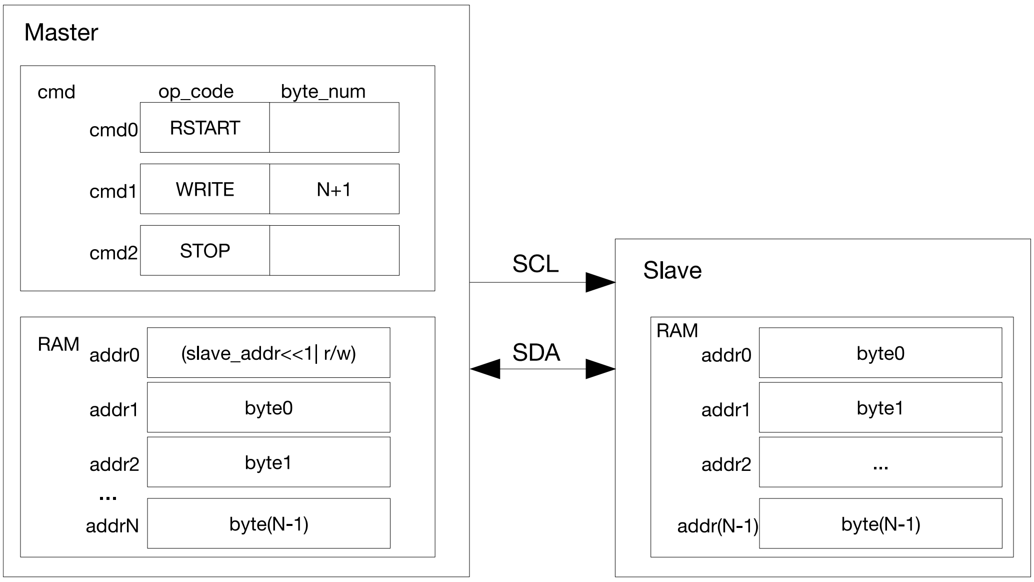

Figure 44.6-1. $I2C_{master}$ Writing to $I2C_{slave}$ with a 7-bit Address

Figure 44.6-1 shows how $I2C_{master}$ writes N bytes of data to $I2C_{slave}$ registers or RAM using 7-bit addressing. As shown in figure 44.6-1, the first byte in the RAM of $I2C_{master}$ is a 7-bit $I2C_{slave}$ address followed by a $R/\overline{W}$ bit. When the $R/\overline{W}$ bit is 0, it indicates a WRITE operation. The remaining bytes are used to store data ready for transfer. The cmd box contains related command sequences.

After the command sequence is configured and data in RAM is ready, $I2C_{master}$ enables the controller and initiates data transfer by setting the I2C_TRANS_START bit. The controller has four steps to take:

1. Wait for SCL to go high, to avoid SCL being used by other masters or slaves.
2. Execute a RSTART command by sending a START bit.
3. Execute a WRITE command by taking N+1 bytes from the RAM in order and sending them to $I2C_{slave}$ in the same order. The first byte is the address of $I2C_{slave}$.
4. Execute a STOP command. Once the $I2C_{master}$ transfers a STOP bit, an I2C_TRANS_COMPLETE_INT interrupt is generated.

44.6.1.2 Configuration Example

1. Configure the timing parameter registers of $I2C_{master}$ and $I2C_{slave}$ according to Section 44.4.7.
2. Set I2C_MS_MODE (master) to 1, and I2C_MS_MODE (slave) to 0.
3. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
4. Configure command registers of $I2C_{master}$.

Command register	op_code	ack_value	ack_exp	ack_check_er	byte_num
I2C_COMMAND0 (master)	RSTART	—	—	—	—
I2C_COMMAND1 (master)	WRITE	—	ack_exp	1	N+1

I2C_COMMAND2 (master)	STOP	—	—	—	—

5. Write the address of $I2C_{slave}$ and data to be sent to TX RAM of $I2C_{master}$ in either FIFO mode or non-FIFO mode according to Section 44.4.10.
6. Write the address of $I2C_{slave}$ to I2C_SLAVE_ADDR (slave) in I2C_SLAVE_ADDR_REG (slave) register.
7. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
8. Write 1 to I2C_TRANS_START (master) and I2C_TRANS_START (slave) to start transfer.
9. $I2C_{slave}$ compares the slave address sent by $I2C_{master}$ with its own address in I2C_SLAVE_ADDR (slave). When ack_check_en (master) in $I2C_{master}$'s WRITE command is 1, $I2C_{master}$ checks ACK value each time it sends a byte. When ack_check_en (master) is 0, $I2C_{master}$ does not check the ACK value and take $I2C_{slave}$ as a matching slave by default.
   - Match: If the received ACK value matches ack_exp (master) (the expected ACK value), $I2C_{master}$ continues data transfer.
   - Not match: If the received ACK value does not match ack_exp, $I2C_{master}$ generates an I2C_NACK_INT (master) interrupt and stops data transfer.
10. $I2C_{master}$ sends data, and determines whether to check ACK value according to ack_check_en (master).
11. If data to be sent (N) is larger than TX FIFO depth, TX RAM of $I2C_{master}$ may wrap around in FIFO mode. For details, please refer to Section 44.4.10.
12. If data to be received (N) is larger than RX FIFO depth, RX RAM of $I2C_{slave}$ may wrap around in FIFO mode. For details, please refer to Section 44.4.10.

    If data to be received (N) is larger than RX FIFO depth, the other way is to enable clock stretching by setting the I2C_SLAVE_SCL_STRETCH_EN (slave), and clearing I2C_RX_FULL_ACK_LEVEL. When RX RAM is full, an I2C_SLAVE_STRETCH_INT (slave) interrupt is generated. In this way, $I2C_{slave}$ can hold SCL low, in exchange for more time to read data. After the software has finished reading, you can set I2C_SLAVE_STRETCH_INT_CLR (slave) to 1 to clear interrupt, and set I2C_SLAVE_SCL_STRETCH_CLR (slave) to release the SCL line.
13. After data transfer completes, $I2C_{master}$ executes the STOP command, and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.

44.6.2 $I2C_{master}$ Writes to $I2C_{slave}$ with a 10-bit Address in One Command Sequence

44.6.2.1 Introduction

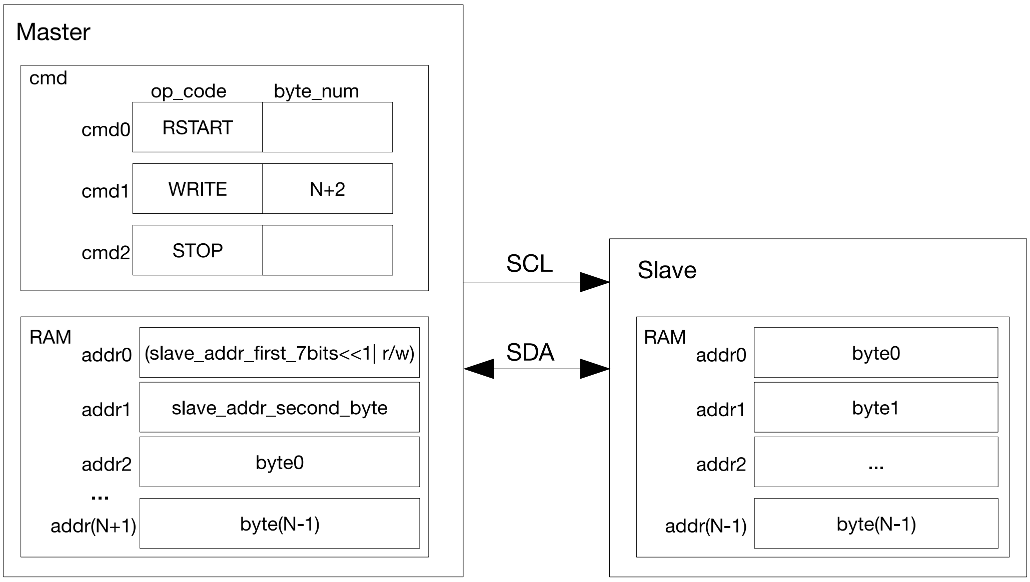

Figure 44.6-2. $I2C_{master}$ Writing to a Slave with a 10-bit Address

Figure 44.6-2 shows how $I2C_{master}$ writes N bytes of data using 10-bit addressing to an I2C slave. The configuration and transfer process is similar to what is described in 44.6.1, except that a 10-bit $I2C_{slave}$ address is formed from two bytes. Since a 10-bit $I2C_{slave}$ address has one more byte than a 7-bit $I2C_{slave}$ address, byte_num and length of data in TX RAM increase by 1 accordingly.

44.6.2.2 Configuration Example

1. Set I2C_MS_MODE (master) to 1, and I2C_MS_MODE (slave) to 0.
2. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
3. Configure command registers of $I2C_{master}$.

Command registers	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	RSTART	—	—	—	—
I2C_COMMAND1 (master)	WRITE	—	ack_exp	1	N+2
I2C_COMMAND2 (master)	STOP	—	—	—	—

4. Configure I2C_SLAVE_ADDR (slave) in I2C_SLAVE_ADDR_REG (slave) as $I2C_{slave}$'s 10-bit address, and set I2C_ADDR_10BIT_EN (slave) to 1 to enable 10-bit addressing.
5. Write the address of $I2C_{slave}$ and data to be sent to TX RAM of $I2C_{master}$. The first byte of the address of $I2C_{slave}$ comprises ((0x78 | I2C_SLAVE_ADDR[9:8])«1) and a $R/\overline{W}$ bit. The second byte of the address of $I2C_{slave}$ is I2C_SLAVE_ADDR[7:0]. These two bytes are followed by data to be sent in FIFO or non-FIFO mode.
6. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
7. Write 1 to I2C_TRANS_START (master) and I2C_TRANS_START (slave) to start transfer.

8. $I2C_{slave}$ compares the slave address sent by $I2C_{master}$ with its own address in I2C_SLAVE_ADDR (slave). When ack_check_en (master) in $I2C_{master}$'s WRITE command is 1, $I2C_{master}$ checks ACK value each time it sends a byte. When ack_check_en (master) is 0, $I2C_{master}$ does not check the ACK value and take $I2C_{slave}$ as a matching slave by default.
   - Match: If the received ACK value matches ack_exp (master) (the expected ACK value), $I2C_{master}$ continues data transfer.
   - Not match: If the received ACK value does not match ack_exp, $I2C_{master}$ generates an I2C_NACK_INT (master) interrupt and stops data transfer.
9. $I2C_{master}$ sends data, and determines whether to check ACK value according to ack_check_en (master).
10. If data to be sent is larger than TX FIFO depth, TX RAM of $I2C_{master}$ may wrap around in FIFO mode. For details, please refer to Section 44.4.10.
11. If data to be received is larger than RX FIFO depth, RX RAM of $I2C_{slave}$ may wrap around in FIFO mode. For details, please refer to Section 44.4.10.

    If data to be received is larger than RX FIFO depth, the other way is to enable clock stretching by setting I2C_SLAVE_SCL_STRETCH_EN (slave), and clearing I2C_RX_FULL_ACK_LEVEL to 0. When RX RAM is full, an I2C_SLAVE_STRETCH_INT (slave) interrupt is generated. In this way, $I2C_{slave}$ can hold SCL low, in exchange for more time to read data. After the software has finished reading, you can set I2C_SLAVE_STRETCH_INT_CLR (slave) to 1 to clear interrupt, and set I2C_SLAVE_SCL_STRETCH_CLR (slave) to release the SCL line.
12. After data transfer completes, $I2C_{master}$ executes the STOP command, and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.

44.6.3 $I2C_{master}$ Writes to $I2C_{slave}$ with Two 7-bit Addresses in One Command Sequence

44.6.3.1 Introduction

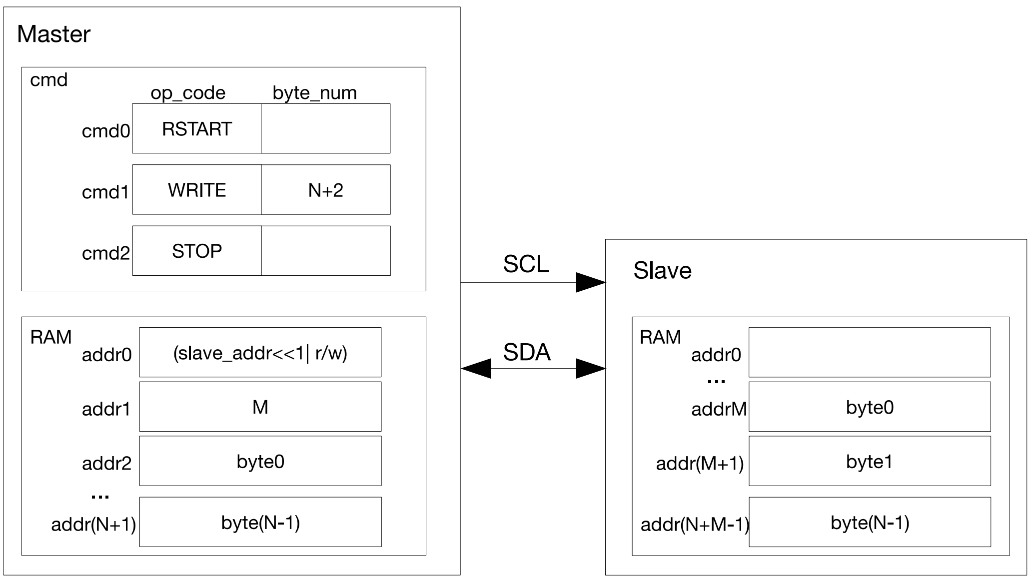

Figure 44.6-3. $I2C_{master}$ Writing to $I2C_{slave}$ with Two 7-bit Addresses

Figure 44.6-3 shows how $I2C_{master}$ writes N bytes of data to $I2C_{slave}$ registers or RAM using 7-bit double addressing. The configuration and transfer process is similar to what is described in Section 44.6.1, except that in 7-bit dual address mode $I2C_{master}$ sends two 7-bit addresses. The first address is the address of an I2C slave, and the second one is $I2C_{slave}$'s memory address (i.e., addrM in Figure 44.6-3). When using double addressing, the slave's RX RAM must be accessed in the non-FIFO mode, and clock stretching by the slave must be disabled. In the non-FIFO mode, the write pointer wraps around to the initial position of the FIFO once its depth is exceeded. As a consequence, data in the slave's RX RAM is overwritten cyclically every 32 bytes. When new data is written to an address that already contains data in the slave, the existing data is directly overwritten without any warning or error indication.

44.6.3.2 Configuration Example

1. Set I2C_MS_MODE (master) to 1, and I2C_MS_MODE (slave) to 0.
2. Set I2C_FIFO_ADDR_CFG_EN (slave) to 1 to enable dual address mode.
3. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
4. Configure command registers of $I2C_{master}$.

Command registers	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	RSTART	—	—	—	—
I2C_COMMAND1 (master)	WRITE	—	ack_exp	1	N+2
I2C_COMMAND2 (master)	STOP	—	—	—	—

5. Write the address of $I2C_{slave}$ and data to be sent to TX RAM of $I2C_{master}$ in FIFO or non-FIFO mode.
6. Write the address of $I2C_{slave}$ to I2C_SLAVE_ADDR (slave) in I2C_SLAVE_ADDR_REG (slave) register.
7. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
8. Write 1 to I2C_TRANS_START (master) and I2C_TRANS_START (slave) to start transfer.
9. $I2C_{slave}$ compares the slave address sent by $I2C_{master}$ with its own address in I2C_SLAVE_ADDR (slave). When ack_check_en (master) in $I2C_{master}$'s WRITE command is 1, $I2C_{master}$ checks ACK value each time it sends a byte. When ack_check_en (master) is 0, $I2C_{master}$ does not check ACK value and takes $I2C_{slave}$ as a matching slave by default.
   - Match: If the received ACK value matches ack_exp (master) (the expected ACK value), $I2C_{master}$ continues data transfer.
   - Not match: If the received ACK value does not match ack_exp, $I2C_{master}$ generates an I2C_NACK_INT (master) interrupt and stops data transfer.
10. $I2C_{slave}$ receives the RX RAM address sent by $I2C_{master}$ and adds the offset.
11. $I2C_{master}$ sends data, and determines whether to check ACK value according to ack_check_en (master).
12. If data to be sent is larger than TX FIFO depth, TX RAM of $I2C_{master}$ may wrap around in FIFO mode. For details, please refer to Section 44.4.10.
13. After data transfer completes, $I2C_{master}$ executes the STOP command, and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.

44.6.4 $I2C_{master}$ Writes to $I2C_{slave}$ with a 7-bit Address in Multiple Command Sequences

44.6.4.1 Introduction

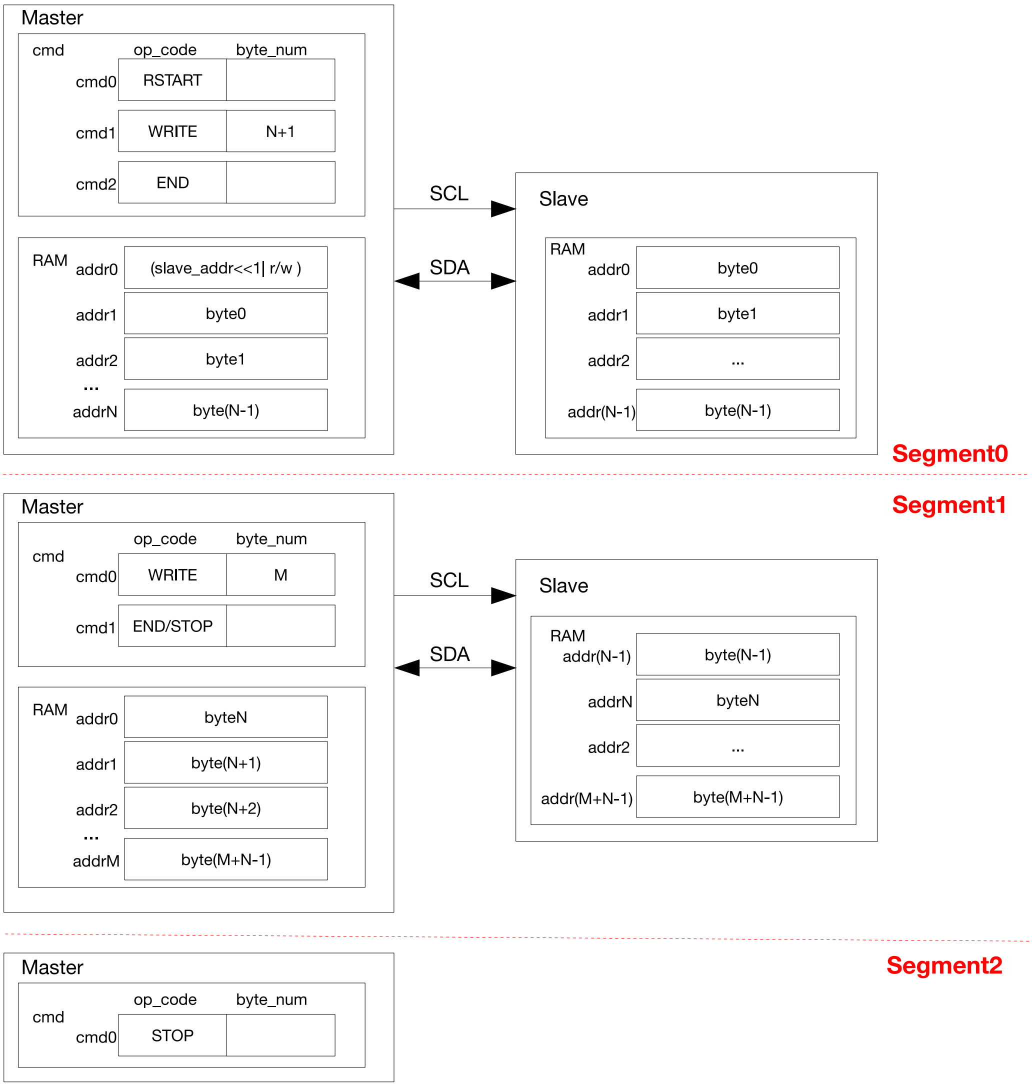

Figure 44.6-4. $I2C_{master}$ Writing to $I2C_{slave}$ with a 7-bit Address in Multiple Sequences

Given that the I2C Controller RAM holds only the size of TX/RX FIFO depth, when data are too large to be processed, it is advised to transmit them in multiple command sequences. At the end of every command sequence is an END command. When the controller executes this END command, SCL will be pulled low, and the software can refresh command sequence registers and the RAM for the next transfer.

Figure 44.6-4 shows how $I2C_{master}$ writes to an I2C slave in two or three segments as an example. For the first segment, the CMD_Controller registers are configured as shown in Segment0. Once data in $I2C_{master}$'s RAM is ready and I2C_TRANS_START is set, $I2C_{master}$ initiates data transfer. After executing the END command, $I2C_{master}$ turns off the SCL clock and pulls SCL low to reserve the bus. Meanwhile, the controller generates an I2C_END_DETECT_INT interrupt.

For the second segment, after detecting the I2C_END_DETECT_INT interrupt, the software refreshes the

CMD_Controller registers, reloads the RAM, and clears this interrupt, as shown in Segment1. If cmd1 in the second segment is a STOP, then data is transmitted to $I2C_{slave}$ in two segments. $I2C_{master}$ resumes data transfer after I2C_TRANS_START is set, and terminates the transfer by sending a STOP bit.

For the third segment, after the second data transfer finishes and an I2C_END_DETECT_INT is detected, the CMD_Controller registers of $I2C_{master}$ are configured as shown in Segment2. Once I2C_TRANS_START is set, $I2C_{master}$ generates a STOP bit and terminates the transfer.

Note that other $I2C_{master}$s will not transact on the bus between two segments. The bus is only released after a STOP command is sent. The I2C controller can be reset by setting I2C_FSM_RST field at any time. This field will later be cleared automatically by hardware.

44.6.4.2 Configuration Example

1. Set I2C_MS_MODE (master) to 1, and I2C_MS_MODE (slave) to 0.
2. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
3. Configure command registers of $I2C_{master}$.

Command registers	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	RSTART	—	—	—	—
I2C_COMMAND1 (master)	WRITE	—	ack_exp	1	N+1
I2C_COMMAND2 (master)	END	—	—	—	—

4. Write the address of $I2C_{slave}$ and data to be sent to TX RAM of $I2C_{master}$ in either FIFO mode or non-FIFO mode according to Section 44.4.10.
5. Write the address of $I2C_{slave}$ to I2C_SLAVE_ADDR (slave) in I2C_SLAVE_ADDR_REG (slave) register
6. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
7. Write 1 to I2C_TRANS_START (master) and I2C_TRANS_START (slave) to start transfer.
8. $I2C_{slave}$ compares the slave address sent by $I2C_{master}$ with its own address in I2C_SLAVE_ADDR (slave). When ack_check_en (master) in $I2C_{master}$'s WRITE command is 1, $I2C_{master}$ checks ACK value each time it sends a byte. When ack_check_en (master) is 0, $I2C_{master}$ does not check ACK value and takes $I2C_{slave}$ as a matching slave by default.
   - Match: If the received ACK value matches ack_exp (master) (the expected ACK value), $I2C_{master}$ continues data transfer.
   - Not match: If the received ACK value does not match ack_exp, $I2C_{master}$ generates an I2C_NACK_INT (master) interrupt and stops data transfer.
9. $I2C_{master}$ sends data, and checks ACK value or not according to ack_check_en (master).
10. After the I2C_END_DETECT_INT (master) interrupt is generated, set I2C_END_DETECT_INT_CLR (master) to 1 to clear this interrupt.
11. Update $I2C_{master}$'s command registers.

Command registers	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	WRITE	—	ack_exp	1	M
I2C_COMMAND1 (master)	END/STOP	—	—	—	—

12. Write M bytes of data to be sent to TX RAM of $I2C_{master}$ in FIFO or non-FIFO mode.
13. Write 1 to I2C_TRANS_START (master) bit to start the transfer and repeat step 9.
14. If the command is a STOP, I2C stops the transfer and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.
15. If the command is an END, repeat step 10.
16. Update $I2C_{master}$'s command registers.

Command registers of $I2C_{master}$	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND1 (master)	STOP	—	—	—	—

17. Write 1 to I2C_TRANS_START (master) bit to start transfer.
18. $I2C_{master}$ executes the STOP command and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.

44.6.5 $I2C_{master}$ Reads $I2C_{slave}$ with a 7-bit Address in One Command Sequence

44.6.5.1 Introduction

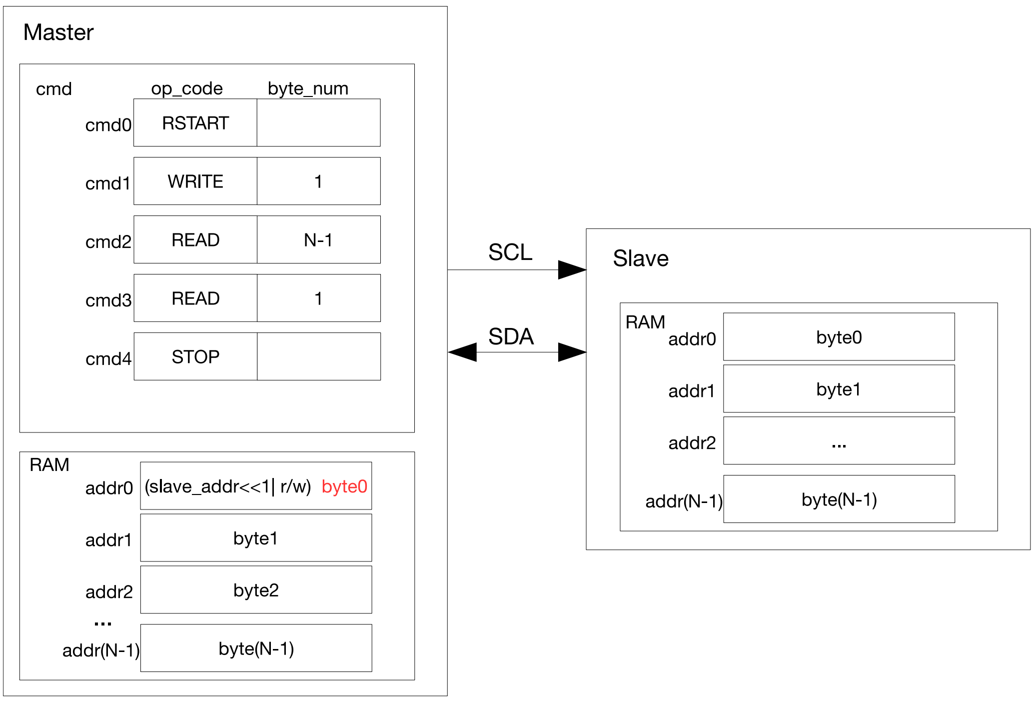

Figure 44.6-5. $I2C_{master}$ Reading $I2C_{slave}$ with a 7-bit Address

Figure 44.6-5 shows how $I2C_{master}$ reads N bytes of data from an I2C slave using 7-bit addressing. cmd1 is a WRITE command, and when this command is executed $I2C_{master}$ sends the address of $I2C_{slave}$. The byte sent comprises a 7-bit $I2C_{slave}$ address and a $R/\overline{W}$ bit. When the $R/\overline{W}$ bit is 1, it indicates a READ operation. If the address of an I2C slave matches the sent address, this matching slave starts sending data to $I2C_{master}$. $I2C_{master}$ generates acknowledgments according to ack_value defined in the READ command upon receiving a byte.

As illustrated in Figure 44.6-5, $I2C_{master}$ executes two READ commands: it generates ACKs for (N-1) bytes of data in cmd2, and a NACK for the last byte of data in cmd 3. This configuration may be changed as required. $I2C_{master}$ writes received data into the controller RAM from addr0, whose original content (the address of $I2C_{slave}$ and a $R/\overline{W}$ bit) is overwritten by byte0 marked red in Figure 44.6-5.

44.6.5.2 Configuration Example

1. Set I2C_MS_MODE (master) to 1, and I2C_MS_MODE (slave) to 0.
2. We recommend setting I2C_SLAVE_SCL_STRETCH_EN (slave) to 1, so that SCL can be held low for more processing time when $I2C_{slave}$ needs to send data. If this bit is not set, the software should write data to be sent to $I2C_{slave}$'s TX RAM before $I2C_{master}$ initiates the transfer. The configuration below is applicable to the scenario where I2C_SLAVE_SCL_STRETCH_EN (slave) is 1.
3. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
4. Configure command registers of $I2C_{master}$.

Command registers of $I2C_{master}$	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	RSTART	—	—	—	—
I2C_COMMAND1 (master)	WRITE	0	0	1	1
I2C_COMMAND2 (master)	READ	0	0	1	N-1
I2C_COMMAND3 (master)	READ	1	0	1	1
I2C_COMMAND4 (master)	STOP	—	—	—	—

5. Write the address of $I2C_{slave}$ to TX RAM of $I2C_{master}$ in either FIFO mode or non-FIFO mode according to Section 44.4.10.
6. Write the address of $I2C_{slave}$ to I2C_SLAVE_ADDR (slave) in I2C_SLAVE_ADDR_REG (slave) register.
7. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
8. Write 1 to I2C_TRANS_START (master) bit to start $I2C_{master}$'s transfer.
9. Start $I2C_{slave}$'s transfer according to Section 44.4.14.
10. $I2C_{slave}$ compares the slave address sent by $I2C_{master}$ with its own address in I2C_SLAVE_ADDR (slave). When ack_check_en (master) in $I2C_{master}$'s WRITE command is 1, $I2C_{master}$ checks ACK value each time it sends a byte. When ack_check_en (master) is 0, $I2C_{master}$ does not check ACK value and takes $I2C_{slave}$ as a matching slave by default.

- Match: If the received ACK value matches ack_exp (master) (the expected ACK value), $I2C_{master}$ continues data transfer.
- Not match: If the received ACK value does not match ack_exp, $I2C_{master}$ generates an I2C_NACK_INT (master) interrupt and stops data transfer.

11. After I2C_SLAVE_STRETCH_INT (slave) is generated, the I2C_STRETCH_CAUSE bit is 0. The address of $I2C_{slave}$ matches the address sent over SDA, and $I2C_{slave}$ needs to send data.
12. Write data to be sent to TX RAM of $I2C_{slave}$ in either FIFO mode or non-FIFO mode according to Section 44.4.10.
13. Set I2C_SLAVE_SCL_STRETCH_CLR (slave) to 1 to release SCL.
14. $I2C_{slave}$ sends data, and $I2C_{master}$ checks ACK value or not according to ack_check_en (master) in the READ command.
15. If data to be read by $I2C_{master}$ is larger than the TX FIFO depth of $I2C_{slave}$, an I2C_SLAVE_STRETCH_INT (slave) interrupt will be generated when TX RAM of $I2C_{slave}$ becomes empty. In this way, $I2C_{slave}$ can hold SCL low, so that software has more time to pad data in TX RAM of $I2C_{slave}$ and read data in RX RAM of $I2C_{master}$. After software has finished reading, you can set I2C_SLAVE_STRETCH_INT_CLR (slave) to 1 to clear interrupt, and set I2C_SLAVE_SCL_STRETCH_CLR (slave) to 1 to release the SCL line.
16. After $I2C_{master}$ has received the last byte of data, set ack_value (master) to 1. $I2C_{slave}$ will stop the transfer once receiving the I2C_NACK_INT interrupt.
17. After data transfer completes, $I2C_{master}$ executes the STOP command, and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.

44.6.6 $I2C_{master}$ Reads $I2C_{slave}$ with a 10-bit Address in One Command Sequence

44.6.6.1 Introduction

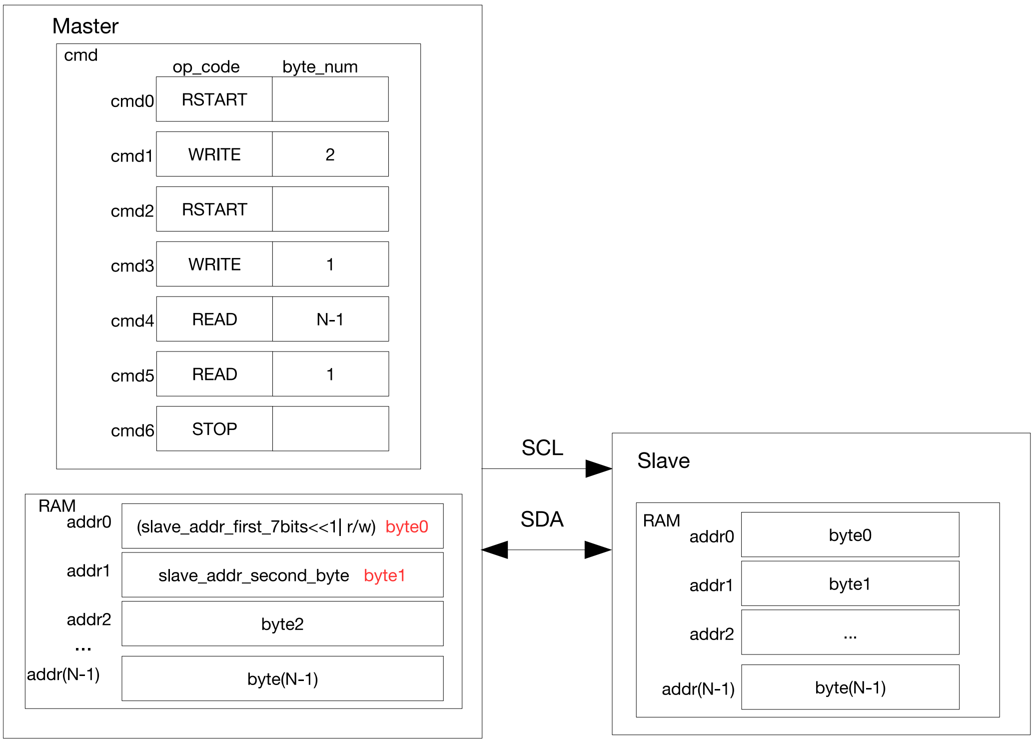

Figure 44.6-6. $I2C_{master}$ Reading $I2C_{slave}$ with a 10-bit Address

Figure 44.6-6 shows how $I2C_{master}$ reads data from an I2C slave using 10-bit addressing. Unlike 7-bit addressing, in 10-bit addressing the WRITE command of the $I2C_{master}$ is formed from two bytes, and correspondingly TX RAM of this master stores a 10-bit address of two bytes. The $R/\overline{W}$ bit in the first byte is 0, which indicates a WRITE operation. After a RSTART condition, $I2C_{master}$ sends the first byte of address again to read data from $I2C_{slave}$, but the $R/\overline{W}$ bit is 1, which indicates a READ operation. The two address bytes can be configured as described in Section 44.6.2.

44.6.6.2 Configuration Example

1. Set I2C_MS_MODE (master) to 1, and I2C_MS_MODE (slave) to 0.
2. We recommend setting I2C_SLAVE_SCL_STRETCH_EN (slave) to 1, so that SCL can be held low for more processing time when $I2C_{slave}$ needs to send data. If this bit is not set, the software should write data to be sent to $I2C_{slave}$'s TX RAM before $I2C_{master}$ initiates the transfer. The configuration below is applicable to a scenario where I2C_SLAVE_SCL_STRETCH_EN (slave) is 1.
3. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
4. Configure command registers of $I2C_{master}$.

Command registers of $I2C_{master}$	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	RSTART	—	—	—	—
I2C_COMMAND1 (master)	WRITE	0	0	1	2

I2C_COMMAND2 (master)	RSTART	—	—	—	—
I2C_COMMAND3 (master)	WRITE	0	0	1	1
I2C_COMMAND4 (master)	READ	0	0	1	N-1
I2C_COMMAND5 (master)	READ	1	0	1	1
I2C_COMMAND6 (master)	STOP	—	—	—	—

5. Configure I2C_SLAVE_ADDR (slave) in I2C_SLAVE_ADDR_REG (slave) as $I2C_{slave}$'s 10-bit address, and set
I2C_ADDR_10BIT_EN (slave) to 1 to enable 10-bit addressing.
6. Write the address of $I2C_{slave}$ and data to be sent to TX RAM of $I2C_{master}$ in either FIFO or non-FIFO mode. The first byte of address comprises ((0x78 | I2C_SLAVE_ADDR[9:8])«1) and a $R/\overline{W}$ bit, which is 1 and indicates a WRITE operation. The second byte of address is I2C_SLAVE_ADDR[7:0]. The third byte is ((0x78 | I2C_SLAVE_ADDR[9:8])«1) and a $R/\overline{W}$ bit, which is 1 and indicates a READ operation.
7. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
8. Write 1 to I2C_TRANS_START (master) to start $I2C_{master}$'s transfer.
9. Start $I2C_{slave}$'s transfer according to Section 44.4.14.
10. $I2C_{slave}$ compares the slave address sent by $I2C_{master}$ with its own address in I2C_SLAVE_ADDR (slave). When ack_check_en (master) in $I2C_{master}$'s WRITE command is 1, $I2C_{master}$ checks ACK value each time it sends a byte. When ack_check_en (master) is 0, $I2C_{master}$ does not check ACK value and takes $I2C_{slave}$ as a matching slave by default.
    - Match: If the received ACK value matches ack_exp (master) (the expected ACK value), $I2C_{master}$ continues data transfer.
    - Not match: If the received ACK value does not match ack_exp, $I2C_{master}$ generates an I2C_NACK_INT (master) interrupt and stops data transfer.
11. $I2C_{master}$ sends a RSTART and the third byte in TX RAM, which is ((0x78 | I2C_SLAVE_ADDR[9:8])«1) and a $R/\overline{W}$ bit that indicates READ.
12. $I2C_{slave}$ repeats step 10. If its address matches the address sent by $I2C_{master}$, $I2C_{slave}$ proceed on to the next steps.
13. After I2C_SLAVE_STRETCH_INT (slave) is generated, the I2C_STRETCH_CAUSE bit is 0. The address of $I2C_{slave}$ matches the address sent over SDA, and $I2C_{slave}$ needs to send data.
14. Write data to be sent to TX RAM of $I2C_{slave}$ in either FIFO mode or non-FIFO mode according to Section 44.4.10.
15. Set I2C_SLAVE_SCL_STRETCH_CLR (slave) to 1 to release SCL.
16. $I2C_{slave}$ sends data, and $I2C_{master}$ checks ACK value or not according to ack_check_en (master) in the READ command.

17. If data to be read by $I2C_{master}$ is larger than the TX FIFO depth of $I2C_{slave}$, an I2C_SLAVE_STRETCH_INT (slave) interrupt will be generated when TX RAM of $I2C_{slave}$ becomes empty. In this way, $I2C_{slave}$ can hold SCL low, so that software has more time to pad data in TX RAM of $I2C_{slave}$ and read data in RX RAM of $I2C_{master}$. After the software has finished reading, you can set I2C_SLAVE_STRETCH_INT_CLR (slave) to 1 to clear interrupt, and set I2C_SLAVE_SCL_STRETCH_CLR (slave) to release the SCL line.

18. After $I2C_{master}$ has received the last byte of data, set ack_value (master) to 1. $I2C_{slave}$ will stop transfer once receiving the I2C_NACK_INT interrupt.

19. After data transfer completes, $I2C_{master}$ executes the STOP command, and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.

44.6.7 $I2C_{master}$ Reads $I2C_{slave}$ with Two 7-bit Addresses in One Command Sequence

44.6.7.1 Introduction

Figure 44.6-7. $I2C_{master}$ Reading N Bytes of Data from addrM of $I2C_{slave}$ with Two 7-bit Addresses

Figure 44.6-7 shows how $I2C_{master}$ reads data from specified addresses in an I2C slave. $I2C_{master}$ sends two bytes of addresses: the first byte is a 7-bit $I2C_{slave}$ address followed by a $R/\overline{W}$ bit, which is 0 and indicates a WRITE; the second byte is $I2C_{slave}$'s memory address. After a RSTART condition, $I2C_{master}$ sends the first byte of address again, but the $R/\overline{W}$ bit is 1 which indicates a READ. Then, $I2C_{master}$ reads data starting from addrM. When double addressing mode is used, the slave's TX RAM must be accessed in non-FIFO mode, and clock stretching by the slave must be disabled. In this configuration, the slave's TX FIFO operates in a non-FIFO manner. Even in the case of an underflow condition where the master attempts to read more data than available, the slave continues to return data corresponding to the current address pointer without generating warnings. The address pointer wraps around to the beginning of the memory space once the end is reached.

44.6.7.2 Configuration Example

1. Set I2C_MS_MODE (master) to 1, and I2C_MS_MODE (slave) to 0.
2. We recommend setting I2C_SLAVE_SCL_STRETCH_EN (slave) to 1, so that SCL can be held low for more processing time when $I2C_{slave}$ needs to send data. If this bit is not set, the software should write data to be sent to $I2C_{slave}$'s TX RAM before $I2C_{master}$ initiates the transfer. The configuration below is applicable to the scenario where I2C_SLAVE_SCL_STRETCH_EN (slave) is 1.
3. Set I2C_FIFO_ADDR_CFG_EN (slave) to 1 to enable dual address mode.
4. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
5. Configure command registers of $I2C_{master}$.

Command registers of $I2C_{master}$	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	RSTART	—	—	—	—
I2C_COMMAND1 (master)	WRITE	0	0	1	2
I2C_COMMAND2 (master)	RSTART	—	—	—	—
I2C_COMMAND3 (master)	WRITE	0	0	1	1
I2C_COMMAND4 (master)	READ	0	0	1	N-1
I2C_COMMAND5 (master)	READ	1	0	1	1
I2C_COMMAND6 (master)	STOP	—	—	—	—

6. Configure I2C_SLAVE_ADDR (slave) in I2C_SLAVE_ADDR_REG (slave) register as $I2C_{slave}$'s 7-bit address, and set I2C_ADDR_10BIT_EN (slave) to 0 to enable 7-bit addressing.
7. Write the address of $I2C_{slave}$ and data to be sent to TX RAM of $I2C_{master}$ in either FIFO or non-FIFO mode according to Section 44.4.10. The first byte of address comprises (I2C_SLAVE_ADDR[6:0])«1) and a $R/\overline{W}$ bit, which is 0 and indicates a WRITE. The second byte of the address is memory address M of $I2C_{slave}$. The third byte is (I2C_SLAVE_ADDR[6:0])«1) and a $R/\overline{W}$ bit, which is 1 and indicates a READ.
8. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
9. Write 1 to I2C_TRANS_START (master) to start $I2C_{master}$'s transfer.
10. Start $I2C_{slave}$'s transfer according to Section 44.4.14.
11. $I2C_{slave}$ compares the slave address sent by $I2C_{master}$ with its own address in I2C_SLAVE_ADDR (slave). When ack_check_en (master) in $I2C_{master}$'s WRITE command is 1, $I2C_{master}$ checks ACK value each time it sends a byte. When ack_check_en (master) is 0, $I2C_{master}$ does not check ACK value and takes $I2C_{slave}$ as a matching slave by default.

- Match: If the received ACK value matches ack_exp (master) (the expected ACK value), $I2C_{master}$ continues data transfer.
- Not match: If the received ACK value does not match ack_exp, $I2C_{master}$ generates an I2C_NACK_INT (master) interrupt and stops data transfer.

12. $I2C_{slave}$ receives memory address sent by $I2C_{master}$ and adds the offset.
13. $I2C_{master}$ sends a RSTART and the third byte in TX RAM, which is ((0x78 | I2C_SLAVE_ADDR[6:0])«1)and an R bit.
14. $I2C_{slave}$ repeats step 11. If its address matches the address sent by $I2C_{master}$, $I2C_{slave}$ proceed on to the next steps.
15. Write data to be sent to TX RAM of $I2C_{slave}$ in non-FIFO mode.
16. $I2C_{slave}$ sends data, and $I2C_{master}$ checks ACK value or not according to ack_check_en (master) in the READ command.
17. After $I2C_{master}$ has received the last byte of data, set ack_value (master) to 1. $I2C_{slave}$ will stop transfer once receiving the I2C_NACK_INT interrupt.
18. After data transfer completes, $I2C_{master}$ executes the STOP command, and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.

44.6.8 $I2C_{master}$ Reads $I2C_{slave}$ with a 7-bit Address in Multiple Command Sequences

44.6.8.1 Introduction

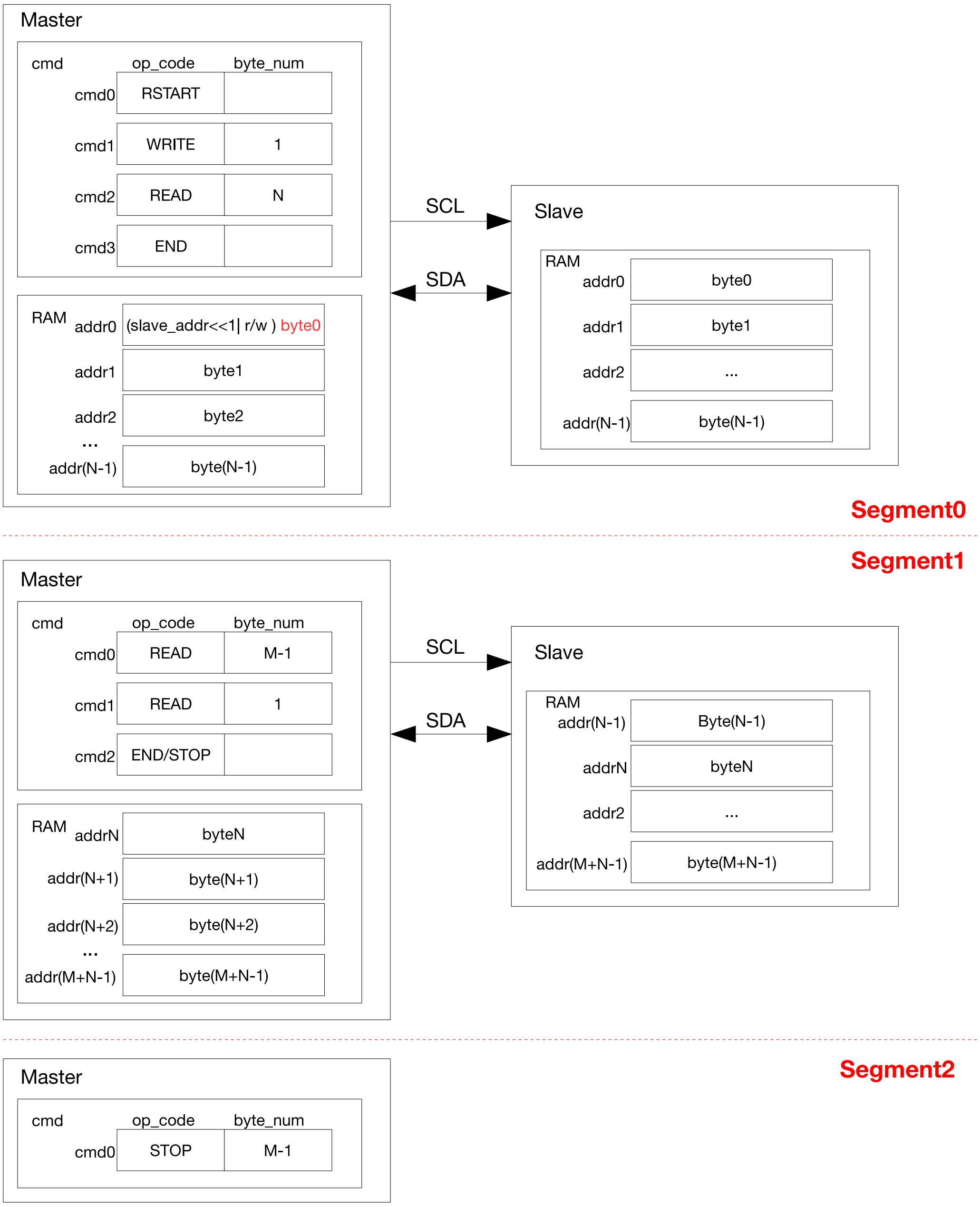

Figure 44.6-8. I2C$_{master}$ Reading I2C$_{slave}$ with a 7-bit Address in Segments

Figure 44.6-8 shows how I2C$_{master}$ reads (N+M) bytes of data from an I2C slave in two/three segments separated by END commands. Configuration procedures are described as follows:

1. The procedures for Segment0 is similar to 44.6-5, except that the last command is an END.
2. Prepare data in the TX RAM of I2C$_{slave}$, and set I2C_TRANS_START to start data transfer. After executing the END command, I2C$_{master}$ refreshes command registers and the RAM as shown in Segment1, and clears the corresponding I2C_END_DETECT_INT interrupt. If cmd2 in Segment1 is a STOP, then data is read from I2C$_{slave}$ in two segments. I2C$_{master}$ resumes data transfer by setting I2C_TRANS_START and terminates the transfer by sending a STOP bit.
3. If cmd2 in Segment1 is an END, then data is read from I2C$_{slave}$ in three segments. After the second data

transfer finishes and an I2C_END_DETECT_INT interrupt is detected, the cmd box is configured as shown in Segment2. Once I2C_TRANS_START is set, $I2C_{master}$ terminates the transfer by sending a STOP bit.

44.6.8.2 Configuration Example

1. Set I2C_MS_MODE (master) to 1, and I2C_MS_MODE (slave) to 0.
2. We recommend setting I2C_SLAVE_SCL_STRETCH_EN (slave) to 1, so that SCL can be held low for more processing time when $I2C_{slave}$ needs to send data. If this bit is not set, the software should write data to be sent to $I2C_{slave}$'s TX RAM before $I2C_{master}$ initiates the transfer. The configuration below is applicable to a scenario where I2C_SLAVE_SCL_STRETCH_EN (slave) is 1.
3. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
4. Configure command registers of $I2C_{master}$.

Command registers of $I2C_{master}$	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	RSTART	—	—	—	—
I2C_COMMAND1 (master)	WRITE	0	0	1	1
I2C_COMMAND2 (master)	READ	0	0	1	N
I2C_COMMAND3 (master)	END	—	—	—	—

5. Write the address of $I2C_{slave}$ to TX RAM of $I2C_{master}$ in FIFO or non-FIFO mode.
6. Write the address of $I2C_{slave}$ to I2C_SLAVE_ADDR (slave) in I2C_SLAVE_ADDR_REG (slave) register.
7. Write 1 to I2C_CONF_UPGATE (master) and I2C_CONF_UPGATE (slave) to synchronize registers.
8. Write 1 to I2C_TRANS_START (master) to start $I2C_{master}$'s transfer.
9. Start $I2C_{slave}$'s transfer according to Section 44.4.14.
10. $I2C_{slave}$ compares the slave address sent by $I2C_{master}$ with its own address in I2C_SLAVE_ADDR (slave). When ack_check_en (master) in $I2C_{master}$'s WRITE command is 1, $I2C_{master}$ checks ACK value each time it sends a byte. When ack_check_en (master) is 0, $I2C_{master}$ does not check the ACK value and takes $I2C_{slave}$ as a matching slave by default.
    - Match: If the received ACK value matches ack_exp (master) (the expected ACK value), $I2C_{master}$ continues data transfer.
    - Not match: If the received ACK value does not match ack_exp, $I2C_{master}$ generates an I2C_NACK_INT (master) interrupt and stops data transfer.
11. After I2C_SLAVE_STRETCH_INT (slave) is generated, the I2C_STRETCH_CAUSE bit is 0. The address of $I2C_{slave}$ matches the address sent over SDA, and $I2C_{slave}$ needs to send data.
12. Write data to be sent to TX RAM of $I2C_{slave}$ in either FIFO mode or non-FIFO mode according to Section 44.4.10.
13. Set I2C_SLAVE_SCL_STRETCH_CLR (slave) to 1 to release SCL.

14. $I2C_{slave}$ sends data, and $I2C_{master}$ checks ACK value or not according to ack_check_en (master) in the READ command.

15. If data to be read by $I2C_{master}$ in one READ command (N or M) is larger than the TX FIFO depth of $I2C_{slave}$, an I2C_SLAVE_STRETCH_INT (slave) interrupt will be generated when TX RAM of $I2C_{slave}$ becomes empty. In this way, $I2C_{slave}$ can hold SCL low so that software has more time to pad data in TX RAM of $I2C_{slave}$ and read data in RX RAM of $I2C_{master}$. After the software has finished reading, you can set I2C_SLAVE_STRETCH_INT_CLR (slave) to 1 to clear interrupt, and set I2C_SLAVE_SCL_STRETCH_CLR (slave) to release the SCL line.

16. Once finishing reading data in the first READ command, $I2C_{master}$ executes the END command and triggers an I2C_END_DETECT_INT (master) interrupt, which is cleared by setting I2C_END_DETECT_INT_CLR (master) to 1.

17. Update $I2C_{master}$'s command registers using one of the following two methods:

Command registers of $I2C_{master}$	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	READ	ack_value	ack_exp	1	M
I2C_COMMAND1 (master)	END	—	—	—	—

Or

Command registers of $I2C_{master}$	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND0 (master)	READ	0	0	1	M-1
I2C_COMMAND1 (master)	READ	1	0	1	1
I2C_COMMAND2 (master)	STOP	—	—	—	—

18. Write M bytes of data to be sent to TX RAM of $I2C_{slave}$. If M is larger than the TX FIFO depth, then repeat step 12 in FIFO or non-FIFO mode.

19. Write 1 to I2C_TRANS_START (master) bit to start the transfer and repeat step 14.

20. If the last command is a STOP, then set ack_value (master) to 1 after $I2C_{master}$ has received the last byte of data. $I2C_{slave}$ stops transfer upon the I2C_NACK_INT interrupt. $I2C_{master}$ executes the STOP command to stop the transfer and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.

21. If the last command is an END, then repeat step 16 and proceed on to the next steps.

22. Update $I2C_{master}$'s command registers.

Command registers of $I2C_{master}$	op_code	ack_value	ack_exp	ack_check_en	byte_num
I2C_COMMAND1 (master)	STOP	—	—	—	—

23. Write 1 to I2C_TRANS_START (master) bit to start transfer.

24. $I2C_{master}$ executes the STOP command to stop transfer, and generates an I2C_TRANS_COMPLETE_INT (master) interrupt.

44.7 Interrupts

ESP32-P4's I2C*n* can generate the I2C*n*_INTR interrupt signal that will be sent to the Interrupt Matrix. There are several internal interrupt sources from I2C*n* that can generate the above interrupt signal(s) as follows:

- I2C_SLAVE_STRETCH_INT: Triggered when one of the four stretching events occurs in slave mode.
- I2C_DET_START_INT: Triggered when the master or the slave detects a START signal.
- I2C_SCL_MAIN_ST_TO_INT: Triggered when the main state machine SCL_MAIN_FSM remains unchanged for over I2C_SCL_MAIN_ST_TO_I2C[23:0] clock cycles.
- I2C_SCL_ST_TO_INT: Triggered when the state machine SCL_FSM remains unchanged for over I2C_SCL_ST_TO_I2C[23:0] clock cycles.
- I2C_RXFIFO_UDF_INT: Triggered when the I2C controller reads RX FIFO via the APB bus, but RX FIFO is empty.
- I2C_TXFIFO_OVF_INT: Triggered when the I2C controller writes TX FIFO via the APB bus, but TX FIFO is full.
- I2C_NACK_INT: Triggered when the ACK value received by the master is not as expected, or when the ACK value received by the slave is 1.
- I2C_TRANS_START_INT: Triggered when the I2C controller sends a START bit.
- I2C_TIME_OUT_INT: Triggered when SCL stays high or low for more than $2^{I2C\_TIME\_OUT\_VALUE}$ clock cycles during data transfer.
- I2C_TRANS_COMPLETE_INT: Triggered when the I2C controller detects a STOP bit.
- I2C_MST_TXFIFO_UDF_INT: Triggered when TX FIFO of the master underflows.
- I2C_ARBITRATION_LOST_INT: Triggered when the SDA's output value does not match its input value while the master's SCL is high.
- I2C_BYTE_TRANS_DONE_INT: Triggered when the I2C controller sends or receives a byte.
- I2C_END_DETECT_INT: Triggered when op_code of the master indicates an END command and an END condition is detected.
- I2C_RXFIFO_OVF_INT: Triggered when RX FIFO of the I2C controller overflows.
- I2C_TXFIFO_WM_INT: I2C TX FIFO watermark interrupt. Triggered when I2C_FIFO_PRT_EN is 1 and the pointers of TX FIFO are less than I2C_TXFIFO_WM_THRHD[4:0].
- I2C_RXFIFO_WM_INT: I2C RX FIFO watermark interrupt. Triggered when I2C_FIFO_PRT_EN is 1 and the pointers of RX FIFO are greater than I2C_RXFIFO_WM_THRHD[4:0].
- I2C_SLAVE_ADDR_UNMATCH_INT: Triggered when the received slave address is inconsistent with the internally configured slave address in slave mode.

> **Note:**
> For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter 12 *Interrupt Matrix* > Section 12.2 *Interrupt Terminology in ESP32-P4*.

Each interrupt source can be configured by a common set of registers that are described in Section *Interrupt Configuration Registers*. The specific registers can be found in Section 44.8.1 *I2C Register Summary*.

44.8 Register Summary

44.8.1 I2C Register Summary

The addresses in this section are relative to the I2C Controller base address provided in Table 7.3-2 in Chapter *7 System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Timing registers			
I2C_SCL_LOW_PERIOD_REG	Configures the low level width of the SCL Clock	0x0000	R/W
I2C_SDA_HOLD_REG	Configures the hold time after a negative SCL edge	0x0030	R/W
I2C_SDA_SAMPLE_REG	Configures the sample time after a positive SCL edge	0x0034	R/W
I2C_SCL_HIGH_PERIOD_REG	Configures the high level width of SCL	0x0038	R/W
I2C_SCL_START_HOLD_REG	Configures the delay between the SDA and SCL negative edge for a start condition	0x0040	R/W
I2C_SCL_RSTART_SETUP_REG	Configures the delay between the positive edge of SCL and the negative edge of SDA	0x0044	R/W
I2C_SCL_STOP_HOLD_REG	Configures the delay after the SCL clock edge for a stop condition	0x0048	R/W
I2C_SCL_STOP_SETUP_REG	Configures the delay between the SDA and SCL rising edge for a stop condition Measurement unit: i2c_sclk	0x004C	R/W
I2C_SCL_ST_TIME_OUT_REG	SCL status time out register	0x0078	R/W
I2C_SCL_MAIN_ST_TIME_OUT_REG	SCL main status time out register	0x007C	R/W
Configuration registers			
I2C_CTR_REG	Transmission setting	0x0004	varies
I2C_TO_REG	Setting time out control for receiving data	0x000C	R/W
I2C_SLAVE_ADDR_REG	Local slave address setting	0x0010	R/W
I2C_FIFO_CONF_REG	FIFO configuration register	0x0018	R/W
I2C_FILTER_CFG_REG	SCL and SDA filter configuration register	0x0050	R/W
I2C_SCL_SP_CONF_REG	Power configuration register	0x0080	varies
I2C_SCL_STRETCH_CONF_REG	Set SCL stretch of I2C slave	0x0084	varies
Status registers			
I2C_SR_REG	Describe I2C work status	0x0008	RO
I2C_FIFO_ST_REG	FIFO status register	0x0014	RO
I2C_DATA_REG	Rx FIFO read data	0x001C	HRO
Interrupt registers			
I2C_INT_RAW_REG	Raw interrupt status	0x0020	R/SS/WTC
I2C_INT_CLR_REG	Interrupt clear bits	0x0024	WT
I2C_INT_ENA_REG	Interrupt enable bits	0x0028	R/W

Name	Description	Address	Access
I2C_INT_STATUS_REG	Status of captured I2C communication events	0x002C	RO
Command registers			
I2C_COMD0_REG	I2C command register 0	0x0058	varies
I2C_COMD1_REG	I2C command register 1	0x005C	varies
I2C_COMD2_REG	I2C command register 2	0x0060	varies
I2C_COMD3_REG	I2C command register 3	0x0064	varies
I2C_COMD4_REG	I2C command register 4	0x0068	varies
I2C_COMD5_REG	I2C command register 5	0x006C	varies
I2C_COMD6_REG	I2C command register 6	0x0070	varies
I2C_COMD7_REG	I2C command register 7	0x0074	varies
Version register			
I2C_DATE_REG	Version register	0x00F8	R/W
Address register			
I2C_TXFIFO_START_ADDR_REG	I2C TX FIFO base address register	0x0100	HRO
I2C_RXFIFO_START_ADDR_REG	I2C RX FIFO base address register	0x0180	HRO

44.8.2 LP_I2C Register Summary

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Timing Registers			
LP_I2C_SCL_LOW_PERIOD_REG	Configures the low level width of the SCL Clock	0x0000	R/W
LP_I2C_SDA_HOLD_REG	Configures the hold time after a negative SCL edge	0x0030	R/W
LP_I2C_SDA_SAMPLE_REG	Configures the sample time after a positive SCL edge	0x0034	R/W
LP_I2C_SCL_HIGH_PERIOD_REG	Configures the high level width of SCL	0x0038	R/W
LP_I2C_SCL_START_HOLD_REG	Configures the delay between the SDA and SCL negative edge for a start condition	0x0040	R/W
LP_I2C_SCL_RSTART_SETUP_REG	Configures the delay between the positive edge of SCL and the negative edge of SDA	0x0044	R/W
LP_I2C_SCL_STOP_HOLD_REG	Configures the delay after the SCL clock edge for a stop condition	0x0048	R/W
LP_I2C_SCL_STOP_SETUP_REG	Configures the delay between the SDA and SCL positive edge for a stop condition	0x004C	R/W
LP_I2C_SCL_ST_TIME_OUT_REG	SCL status time out register	0x0078	R/W
LP_I2C_SCL_MAIN_ST_TIME_OUT_REG	SCL main status time out register	0x007C	R/W
Configuration Registers			
LP_I2C_CTR_REG	Transmission setting	0x0004	varies
LP_I2C_TO_REG	Setting time out control for receiving data	0x000C	R/W
LP_I2C_FIFO_CONF_REG	FIFO configuration register	0x0018	R/W
LP_I2C_FILTER_CFG_REG	SCL and SDA filter configuration register	0x0050	R/W
LP_I2C_SCL_SP_CONF_REG	Power configuration register	0x0080	varies

Name	Description	Address	Access
Status Registers			
LP_I2C_SR_REG	Describe I2C work status	0x0008	RO
LP_I2C_FIFO_ST_REG	FIFO status register	0x0014	RO
LP_I2C_DATA_REG	Rx FIFO read data	0x001C	RO
Interrupt Registers			
LP_I2C_INT_RAW_REG	Raw interrupt status	0x0020	R/SS/WTC
LP_I2C_INT_CLR_REG	Interrupt clear bits	0x0024	WT
LP_I2C_INT_ENA_REG	Interrupt enable bits	0x0028	R/W
LP_I2C_INT_STATUS_REG	Status of captured I2C communication events	0x002C	RO
Command Registers			
LP_I2C_COMD0_REG	I2C command register 0	0x0058	varies
LP_I2C_COMD1_REG	I2C command register 1	0x005C	varies
LP_I2C_COMD2_REG	I2C command register 2	0x0060	varies
LP_I2C_COMD3_REG	I2C command register 3	0x0064	varies
LP_I2C_COMD4_REG	I2C command register 4	0x0068	varies
LP_I2C_COMD5_REG	I2C command register 5	0x006C	varies
LP_I2C_COMD6_REG	I2C command register 6	0x0070	varies
LP_I2C_COMD7_REG	I2C command register 7	0x0074	varies
Version Register			
LP_I2C_DATE_REG	Version register	0x00F8	R/W
Address Registers			
LP_I2C_TXFIFO_START_ADDR_REG	I2C TXFIFO base address register	0x0100	HRO
LP_I2C_RXFIFO_START_ADDR_REG	I2C RXFIFO base address register	0x0180	HRO

44.9 Registers

44.9.1 I2C Registers

The addresses in this section are relative to the I2C Controller base address provided in Table 7.3-2 in Chapter *7 System and Memory*.

For how to program reserved fields, please refer to Section Programming Reserved Register Field.

Register 44.1. I2C_SCL_LOW_PERIOD_REG (0x0000)

	(reserved)	I2C_SCL_LOW_PERIOD	
Bits	31 … 9	8 … 0	
	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_SCL_LOW_PERIOD Configures the low level width of the SCL Clock in master mode.
Measurement unit: I2C_SCLK clock cycles
(R/W)

Register 44.2. I2C_SDA_HOLD_REG (0x0030)

	(reserved)	I2C_SDA_HOLD_TIME	
Bits	31 … 9	8 … 0	
	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_SDA_HOLD_TIME Configures the time to hold the data after the falling edge of SCL.
Measurement unit: I2C_SCLK clock cycles
(R/W)

Register 44.3. I2C_SDA_SAMPLE_REG (0x0034)

(reserved)	I2C_SDA_SAMPLE_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_SDA_SAMPLE_TIME Configures the time for sampling SDA.

Measurement unit: I2C_SCLK clock cycles

(R/W)

Register 44.4. I2C_SCL_HIGH_PERIOD_REG (0x0038)

(reserved)	I2C_SCL_WAIT_HIGH_PERIOD	I2C_SCL_HIGH_PERIOD	
31 … 16	15 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	Reset

I2C_SCL_HIGH_PERIOD Configures for how long SCL remains high in master mode.

Measurement unit: I2C_SCLK clock cycles

(R/W)

I2C_SCL_WAIT_HIGH_PERIOD Configures the SCL_FSM's waiting period for SCL high level in master mode.

Measurement unit: I2C_SCLK clock cycles

(R/W)

Register 44.5. I2C_SCL_START_HOLD_REG (0x0040)

(reserved)	I2C_SCL_START_HOLD_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	8	Reset

I2C_SCL_START_HOLD_TIME Configures the time between the falling edge of SDA and the falling edge of SCL for a START condition.
Measurement unit: I2C_SCLK clock cycles
(R/W)

Register 44.6. I2C_SCL_RSTART_SETUP_REG (0x0044)

(reserved)	I2C_SCL_RSTART_SETUP_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	8	Reset

I2C_SCL_RSTART_SETUP_TIME Configures the time between the positive edge of SCL and the negative edge of SDA for a RESTART condition.
Measurement unit: I2C_SCLK clock cycles
(R/W)

Register 44.7. I2C_SCL_STOP_HOLD_REG (0x0048)

(reserved)	I2C_SCL_STOP_HOLD_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	8	Reset

I2C_SCL_STOP_HOLD_TIME Configures the delay after the STOP condition.
Measurement unit: I2C_SCLK clock cycles
(R/W)

Register 44.8. I2C_SCL_STOP_SETUP_REG (0x004C)

(reserved)	I2C_SCL_STOP_SETUP_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	8	Reset

I2C_SCL_STOP_SETUP_TIME Configures the time between the rising edge of SCL and the rising edge of SDA.

Measurement unit: I2C_SCLK clock cycles

(R/W)

Register 44.9. I2C_SCL_ST_TIME_OUT_REG (0x0078)

(reserved)	I2C_SCL_ST_TO_I2C	
31 … 5	4 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0x10	Reset

I2C_SCL_ST_TO_I2C Configures the threshold value of SCL_FSM state unchanged period. It should be no more than 23, more than 1.

$$\text{Maximum Clock Cycle Threshold} = 2^{\text{I2C\_SCL\_ST\_TO\_I2C}+1} - 1$$

Measurement unit: I2C_SCLK clock cycles

(R/W)

Register 44.10. I2C_SCL_MAIN_ST_TIME_OUT_REG (0x007C)

(reserved)	I2C_SCL_MAIN_ST_TO_I2C	
31 … 5	4 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0x10	Reset

I2C_SCL_MAIN_ST_TO_I2C Configures the threshold value of SCL_MAIN_FSM state unchanged period. It should be no more than 23.
Measurement unit: I2C_SCLK clock cycles
(R/W)

Register 44.11. I2C_CTR_REG (0x0004)

Bits	31–15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Field	(reserved)	I2C_ADDR_BROADCASTING_EN	I2C_ADDR_10BIT_RW_CHECK_EN	I2C_SLV_TX_AUTO_START_EN	I2C_CONF_UPGATE	I2C_FSM_RST	I2C_ARBITRATION_EN	I2C_CLK_EN	I2C_RX_LSB_FIRST	I2C_TX_LSB_FIRST	I2C_TRANS_START	I2C_MS_MODE	I2C_RX_FULL_ACK_LEVEL	I2C_SAMPLE_SCL_LEVEL	I2C_SCL_FORCE_OUT	I2C_SDA_FORCE_OUT	
Value	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	Reset

I2C_SDA_FORCE_OUT Configures the SDA output mode.

0: Open drain output

1: Direct output

(R/W)

I2C_SCL_FORCE_OUT Configures the SCL output mode.

0: Open drain output

1: Direct output

(R/W)

I2C_SAMPLE_SCL_LEVEL Configures the sample mode for SDA.

0: Sample SDA data on the SCL high level

1: Sample SDA data on the SCL low level

(R/W)

I2C_RX_FULL_ACK_LEVEL Configures the ACK value that needs to be sent by the master when the received RX RAM has reached the threshold.

(R/W)

I2C_MS_MODE Configures the module as an I2C Master or Slave.

0: Slave

1: Master

(R/W)

I2C_TRANS_START Configures whether the slave starts sending the data in TX FIFO.

0: No effect

1: Start

(WT)

I2C_TX_LSB_FIRST Configures to control the sending order for data needing to be sent.

0: send data from the most significant bit

1: send data from the least significant bit

(R/W)

I2C_RX_LSB_FIRST Configures to control the storage order for received data.

0: receive data from the most significant bit

1: receive data from the least significant bit

(R/W)

Continued on the next page...

Register 44.11. I2C_CTR_REG (0x0004)

Continued from the previous page...

I2C_CLK_EN Configures whether to gate clock signal for registers.
0: Support clock only when registers are read or written to by software
1: Force clock on for registers
(R/W)

I2C_ARBITRATION_EN Configures to enable I2C bus arbitration detection.
0: No effect
1: Enable
(R/W)

I2C_FSM_RST Configures to reset the SCL_FSM.
0: No effect
1: Reset (WT)

I2C_CONF_UPGATE Configures this bit for synchronization.
0: No effect
1: Synchronize (WT)

I2C_SLV_TX_AUTO_START_EN Configures to enable slave to send data automatically
0: Disable
1: Enable
(R/W)

I2C_ADDR_10BIT_RW_CHECK_EN Configures to check if the R/W bit of 10-bit addressing consists with I2C protocol.
0: Not check
1: Check (R/W)

I2C_ADDR_BROADCASTING_EN Configures to support the 7-bit general call function.
0: Not support
1: Support
(R/W)

Register 44.12. I2C_TO_REG (0x000C)

Field	(reserved)	I2C_TIME_OUT_EN	I2C_TIME_OUT_VALUE
Bits	31–6	5	4–0
Reset	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0x10

I2C_TIME_OUT_VALUE Configures the timeout threshold period for SCL stuck at high or low level.
The actual period is 2^(I2C_TIME_OUT_VALUE+1)−1.
Measurement unit: I2C_SCLK clock cycles
(R/W)

I2C_TIME_OUT_EN Configures to enable time out control.
0: No effect
1: Enable
(R/W)

Register 44.13. I2C_SLAVE_ADDR_REG (0x0010)

Field	I2C_ADDR_10BIT_EN	(reserved)	I2C_SLAVE_ADDR
Bits	31	30–15	14–0
Reset	0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0

I2C_SLAVE_ADDR Configure the slave address of I2C Slave.
(R/W)

I2C_ADDR_10BIT_EN Configures to enable the slave 10-bit addressing mode in master mode.
0: No effect
1: Enable
(R/W)

Register 44.14. I2C_FIFO_CONF_REG (0x0018)

(reserved)	I2C_FIFO_PRT_EN	I2C_TX_FIFO_RST	I2C_RX_FIFO_RST	I2C_FIFO_ADDR_CFG_EN	I2C_NONFIFO_EN	I2C_TXFIFO_WM_THRHD	I2C_RXFIFO_WM_THRHD	
31 … 15	14	13	12	11	10	9 … 5	4 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	1	0	0	0	0	0x4	0xb	Reset

I2C_RXFIFO_WM_THRHD Configures the watermark threshold of RX FIFO in non-FIFO access mode. When I2C_FIFO_PRT_EN is 1 and RX FIFO counter is bigger than I2C_RXFIFO_WM_THRHD[4:0], I2C_RXFIFO_WM_INT_RAW bit will be valid. (R/W)

I2C_TXFIFO_WM_THRHD Configures the watermark threshold of TX FIFO in non-FIFO access mode. When I2C_FIFO_PRT_EN is 1 and TC FIFO counter is bigger than I2C_TXFIFO_WM_THRHD[4:0], I2C_TXFIFO_WM_INT_RAW bit will be valid. (R/W)

I2C_NONFIFO_EN Configures to enable APB non-FIFO access. (R/W)

I2C_FIFO_ADDR_CFG_EN Configures the slave to enable dual address mode. When this mode is enabled, the byte received after the I2C address byte represents the offset address in the I2C Slave RAM.
0: Disable
1: Enable
(R/W)

I2C_RX_FIFO_RST Configures to reset RX FIFO.
0: No effect
1: Reset (R/W)

I2C_TX_FIFO_RST Configures to reset TX FIFO.
0: No effect
1: Reset (R/W)

I2C_FIFO_PRT_EN Configures to enable FIFO pointer in non-FIFO access mode. This bit controls the valid bits and the TX/RX FIFO overflow, underflow, full and empty interrupts.
0: No effect
1: Enable
(R/W)

Register 44.15. I2C_FILTER_CFG_REG (0x0050)

(reserved)	I2C_SDA_FILTER_EN	I2C_SCL_FILTER_EN	I2C_SDA_FILTER_THRES	I2C_SCL_FILTER_THRES	
31 … 10	9	8	7 … 4	3 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	1	1	0	0	Reset

I2C_SCL_FILTER_THRES Configures the threshold pulse width to be filtered on SCL. When a pulse on the SCL input has smaller width than this register value, the I2C controller will ignore that pulse. Measurement unit: I2C_SCLK clock cycles
(R/W)

I2C_SDA_FILTER_THRES Configures the threshold pulse width to be filtered on SDA. When a pulse on the SDA input has smaller width than this register value, the I2C controller will ignore that pulse. Measurement unit: I2C_SCLK clock cycles
(R/W)

I2C_SCL_FILTER_EN Configures to enable the filter function for SCL.
0: No effect
1: Enable
(R/W)

I2C_SDA_FILTER_EN Configures to enable the filter function for SDA.
0: No effect
1: Enable
(R/W)

Register 44.16. I2C_SCL_SP_CONF_REG (0x0080)

	(reserved)	I2C_SDA_PD_EN	I2C_SCL_PD_EN	I2C_SCL_RST_SLV_NUM	I2C_SCL_RST_SLV_EN
Bits	31 … 8	7	6	5 … 1	0
Reset	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0

I2C_SCL_RST_SLV_EN Configures to send out SCL pulses when I2C master is IDLE. The number of pulses equals to I2C_SCL_RST_SLV_NUM[4:0].
0: Invalid
1: Send out SCL pulses
(R/W/SC)

I2C_SCL_RST_SLV_NUM Configure the pulses of SCL generated in I2C master mode.
Valid when I2C_SCL_RST_SLV_EN is 1.
Measurement unit: I2C_SCLK clock cycles
(R/W)

I2C_SCL_PD_EN Configures to power down the I2C output SCL line.
0: Not power down.
1: Not work and power down.
Valid only when I2C_SCL_FORCE_OUT is 1. (R/W)

I2C_SDA_PD_EN Configures to power down the I2C output SDA line.
0: Not power down.
1: Not work and power down.
Valid only when I2C_SDA_FORCE_OUT is 1. (R/W)

Register 44.17. I2C_SCL_STRETCH_CONF_REG (0x0084)

(reserved)	I2C_SLAVE_BYTE_ACK_LVL	I2C_SLAVE_BYTE_ACK_CTL_EN	I2C_SLAVE_SCL_STRETCH_CLR	I2C_SLAVE_SCL_STRETCH_EN	I2C_STRETCH_PROTECT_NUM	
31 ... 14	13	12	11	10	9 ... 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0	0	Reset

I2C_STRETCH_PROTECT_NUM Configures the time period to release the SCL line from stretching to avoid timing violation. Usually it should be larger than the SDA setup time.
Measurement unit: I2C_SCLK clock cycles
(R/W)

I2C_SLAVE_SCL_STRETCH_EN Configures to enable slave SCL stretch function. The SCL output line will be stretched low when I2C_SLAVE_SCL_STRETCH_EN is 1 and a stretch event happens. The stretch cause can be seen in I2C_STRETCH_CAUSE.
0: Disable
1: Enable
(R/W)

I2C_SLAVE_SCL_STRETCH_CLR Configures to clear the I2C slave SCL stretch function.
0: No effect
1: Clear
(WT)

I2C_SLAVE_BYTE_ACK_CTL_EN Configures to enable the function for the slave to control ACK level.
0: Disable
1: Enable
(R/W)

I2C_SLAVE_BYTE_ACK_LVL Configures the ACK level when slave controlling ACK level function is enabled.
0: Low level
1: High level
(R/W)

Register 44.18. I2C_SR_REG (0x0008)

(reserved)	I2C_SCL_STATE_LAST	(reserved)	I2C_SCL_MAIN_STATE_LAST	I2C_TXFIFO_CNT	(reserved)	I2C_STRETCH_CAUSE	I2C_RXFIFO_CNT	(reserved)	I2C_SLAVE_ADDRESSED	I2C_BUS_BUSY	I2C_ARB_LOST	(reserved)	I2C_SLAVE_RW	I2C_RESP_REC	
31	30 28	27	26 24	23 18	17 16	15 14	13 8	7 6	5	4	3	2	1	0	
0	0	0	0	0	0 0	0x3	0	0 0	0	0	0	0	0	0	Reset

I2C_RESP_REC Represents the received ACK value in master mode or slave mode.
0: ACK
1: NACK. (RO)

I2C_SLAVE_RW Represents the transfer direction in slave mode.
0: Master writes to slave. 1: Master reads from slave
(RO)

I2C_ARB_LOST Represents whether the I2C controller loses control of SCL line.
0: No arbitration lost
1: Arbitration lost
(RO)

I2C_BUS_BUSY Represents the I2C bus state.
0: The I2C bus is in an idle state.
1: The I2C bus is busy transferring data
(RO)

I2C_SLAVE_ADDRESSED Represents whether the address sent by the master is equal to the address of the slave.
Valid only when the module is configured as an I2C Slave.
0: Not equal
1: Equal
(RO)

I2C_RXFIFO_CNT Represents the number of data bytes received in RAM. (RO)

I2C_STRETCH_CAUSE Represents the cause of SCL clocking stretching in slave mode.
0: Stretching SCL low when the master starts to read data.
1: Stretching SCL low when I2C TX FIFO is empty in slave mode.
2: Stretching SCL low when I2C RX FIFO is full in slave mode.
3: Invalid
(RO)

I2C_TXFIFO_CNT Represents the number of data bytes to be sent. (RO)

Continued on the next page...

Register 44.18. I2C_SR_REG (0x0008)

Continued from the previous page...

I2C_SCL_MAIN_STATE_LAST Represents the states of the I2C module state machine.
- 0: Idle
- 1: Address shift
- 2: ACK address
- 3: Rx data
- 4: Tx data
- 5: Send ACK
- 6: Wait ACK
- 7: Invalid

(RO)

I2C_SCL_STATE_LAST Represents the states of the state machine used to produce SCL.
- 0: Idle
- 1: Start
- 2: Negative edge
- 3: Low
- 4: Positive edge
- 5: High
- 6: Stop
- 7: Invalid

(RO)

Register 44.19. I2C_FIFO_ST_REG (0x0014)

(reserved)	I2C_SLAVE_RW_POINT	(reserved)	I2C_TXFIFO_WADDR	I2C_TXFIFO_RADDR	I2C_RXFIFO_WADDR	I2C_RXFIFO_RADDR	
31 30	29 22	21 20	19 15	14 10	9 5	4 0	
0 0	0	0 0	0	0	0	0	Reset

I2C_RXFIFO_RADDR Represents the offset address of the APB reading from RX FIFO. (RO)

I2C_RXFIFO_WADDR Represents the offset address of the I2C module receiving data and writing to RX FIFO. (RO)

I2C_TXFIFO_RADDR Represents the offset address of the I2C module reading from TX FIFO. (RO)

I2C_TXFIFO_WADDR Represents the offset address of the APB bus writing to TX FIFO. (RO)

I2C_SLAVE_RW_POINT Represents the offset address in the I2C Slave RAM addressed by I2C Master when in I2C slave mode. (RO)

Register 44.20. I2C_DATA_REG (0x001C)

Bits	Field	Reset
31:8	(reserved)	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
7:0	I2C_FIFO_RDATA	0

I2C_FIFO_RDATA Represents the value of RX FIFO read data. (RO)

Register 44.21. I2C_INT_RAW_REG (0x0020)

Bits	Field	Reset
31:19	(reserved)	0 0 0 0 0 0 0 0 0 0 0 0 0
18	I2C_SLAVE_ADDR_UNMATCH_INT_RAW	0
17	I2C_GENERAL_CALL_INT_RAW	0
16	I2C_SLAVE_STRETCH_INT_RAW	0
15	I2C_DET_START_INT_RAW	0
14	I2C_SCL_MAIN_ST_TO_INT_RAW	0
13	I2C_SCL_ST_TO_INT_RAW	0
12	I2C_RXFIFO_UDF_INT_RAW	0
11	I2C_TXFIFO_OVF_INT_RAW	0
10	I2C_NACK_INT_RAW	0
9	I2C_TRANS_START_INT_RAW	0
8	I2C_TIME_OUT_INT_RAW	0
7	I2C_TRANS_COMPLETE_INT_RAW	0
6	I2C_MST_TXFIFO_UDF_INT_RAW	0
5	I2C_ARBITRATION_LOST_INT_RAW	0
4	I2C_BYTE_TRANS_DONE_INT_RAW	0
3	I2C_END_DETECT_INT_RAW	0
2	I2C_RXFIFO_OVF_INT_RAW	0
1	I2C_TXFIFO_WM_INT_RAW	1
0	I2C_RXFIFO_WM_INT_RAW	0

I2C_RXFIFO_WM_INT_RAW The raw interrupt status of I2C_RXFIFO_WM_INT interrupt. (R/SS/WTC)

I2C_TXFIFO_WM_INT_RAW The raw interrupt status of I2C_TXFIFO_WM_INT interrupt. (R/SS/WTC)

I2C_RXFIFO_OVF_INT_RAW The raw interrupt status of I2C_RXFIFO_OVF_INT interrupt. (R/SS/WTC)

I2C_END_DETECT_INT_RAW The raw interrupt status of the I2C_END_DETECT_INT interrupt. (R/SS/WTC)

I2C_BYTE_TRANS_DONE_INT_RAW The raw interrupt status of the I2C_END_DETECT_INT interrupt. (R/SS/WTC)

I2C_ARBITRATION_LOST_INT_RAW The raw interrupt status of the I2C_ARBITRATION_LOST_INT interrupt. (R/SS/WTC)

I2C_MST_TXFIFO_UDF_INT_RAW The raw interrupt status of I2C_TRANS_COMPLETE_INT interrupt. (R/SS/WTC)

I2C_TRANS_COMPLETE_INT_RAW The raw interrupt status of the I2C_TRANS_COMPLETE_INT interrupt. (R/SS/WTC)

Continued on the next page...

Register 44.21. I2C_INT_RAW_REG (0x0020)

Continued from the previous page...

I2C_TIME_OUT_INT_RAW The raw interrupt status of the I2C_TIME_OUT_INT interrupt. (R/SS/WTC)

I2C_TRANS_START_INT_RAW The raw interrupt status of the I2C_TRANS_START_INT interrupt. (R/SS/WTC)

I2C_NACK_INT_RAW The raw interrupt status of I2C_SLAVE_STRETCH_INT interrupt. (R/SS/WTC)

I2C_TXFIFO_OVF_INT_RAW The raw interrupt status of I2C_TXFIFO_OVF_INT interrupt. (R/SS/WTC)

I2C_RXFIFO_UDF_INT_RAW The raw interrupt status of I2C_RXFIFO_UDF_INT interrupt. (R/SS/WTC)

I2C_SCL_ST_TO_INT_RAW The raw interrupt status of I2C_SCL_ST_TO_INT interrupt. (R/SS/WTC)

I2C_SCL_MAIN_ST_TO_INT_RAW The raw interrupt status of I2C_SCL_MAIN_ST_TO_INT interrupt. (R/SS/WTC)

I2C_DET_START_INT_RAW The raw interrupt status of I2C_DET_START_INT interrupt. (R/SS/WTC)

I2C_SLAVE_STRETCH_INT_RAW The raw interrupt status of I2C_SLAVE_STRETCH_INT interrupt. (R/SS/WTC)

I2C_GENERAL_CALL_INT_RAW The raw interrupt status of I2C_GENARAL_CALL_INT interrupt. (R/SS/WTC)

I2C_SLAVE_ADDR_UNMATCH_INT_RAW The raw interrupt status of I2C_SLAVE_ADDR_UNMATCH_INT_RAW interrupt. (R/SS/WTC)

Register 44.22. I2C_INT_CLR_REG (0x0024)

Bits	31–19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	(reserved)	I2C_SLAVE_ADDR_UNMATCH_INT_CLR	I2C_GENERAL_CALL_INT_CLR	I2C_SLAVE_STRETCH_INT_CLR	I2C_DET_START_INT_CLR	I2C_SCL_MAIN_ST_TO_INT_CLR	I2C_SCL_ST_TO_INT_CLR	I2C_RXFIFO_UDF_INT_CLR	I2C_TXFIFO_OVF_INT_CLR	I2C_NACK_INT_CLR	I2C_TRANS_START_INT_CLR	I2C_TIME_OUT_INT_CLR	I2C_TRANS_COMPLETE_INT_CLR	I2C_MST_TXFIFO_UDF_INT_CLR	I2C_ARBITRATION_LOST_INT_CLR	I2C_BYTE_TRANS_DONE_INT_CLR	I2C_END_DETECT_INT_CLR	I2C_RXFIFO_OVF_INT_CLR	I2C_TXFIFO_WM_INT_CLR	I2C_RXFIFO_WM_INT_CLR
Reset	0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

I2C_RXFIFO_WM_INT_CLR Write 1 to clear I2C_RXFIFO_WM_INT interrupt. (WT)

I2C_TXFIFO_WM_INT_CLR Write 1 to clear I2C_TXFIFO_WM_INT interrupt. (WT)

I2C_RXFIFO_OVF_INT_CLR Write 1 to clear I2C_RXFIFO_OVF_INT interrupt. (WT)

I2C_END_DETECT_INT_CLR Write 1 to clear the I2C_END_DETECT_INT interrupt. (WT)

I2C_BYTE_TRANS_DONE_INT_CLR Write 1 to clear the I2C_BYTE_TRANS_DONE_INT interrupt. (WT)

I2C_ARBITRATION_LOST_INT_CLR Write 1 to clear the I2C_ARBITRATION_LOST_INT interrupt. (WT)

I2C_MST_TXFIFO_UDF_INT_CLR Write 1 to clear I2C_MST_TXFIFO_UDF_INT interrupt. (WT)

I2C_TRANS_COMPLETE_INT_CLR Write 1 to clear the I2C_TRANS_COMPLETE_INT interrupt. (WT)

I2C_TIME_OUT_INT_CLR Write 1 to clear the I2C_TIME_OUT_INT interrupt. (WT)

I2C_TRANS_START_INT_CLR Write 1 to clear the I2C_TRANS_START_INT interrupt. (WT)

I2C_NACK_INT_CLR Write 1 to clear I2C_NACK_INT interrupt. (WT)

I2C_TXFIFO_OVF_INT_CLR Write 1 to clear I2C_TXFIFO_OVF_INT interrupt. (WT)

I2C_RXFIFO_UDF_INT_CLR Write 1 to clear I2C_RXFIFO_UDF_INT interrupt. (WT)

I2C_SCL_ST_TO_INT_CLR Write 1 to clear I2C_SCL_ST_TO_INT interrupt. (WT)

I2C_SCL_MAIN_ST_TO_INT_CLR Write 1 to clear I2C_SCL_MAIN_ST_TO_INT interrupt. (WT)

I2C_DET_START_INT_CLR Write 1 to clear I2C_DET_START_INT interrupt. (WT)

I2C_SLAVE_STRETCH_INT_CLR Write 1 to clear I2C_SLAVE_STRETCH_INT interrupt. (WT)

I2C_GENERAL_CALL_INT_CLR Write 1 to clear I2C_GENARAL_CALL_INT interrupt. (WT)

I2C_SLAVE_ADDR_UNMATCH_INT_CLR Write 1 to clear I2C_SLAVE_ADDR_UNMATCH_INT_RAW interrupt. (WT)

Register 44.23. I2C_INT_ENA_REG (0x0028)

Field	(reserved)	I2C_SLAVE_ADDR_UNMATCH_INT_ENA	I2C_GENERAL_CALL_INT_ENA	I2C_SLAVE_STRETCH_INT_ENA	I2C_DET_START_INT_ENA	I2C_SCL_MAIN_ST_TO_INT_ENA	I2C_SCL_ST_TO_INT_ENA	I2C_RXFIFO_UDF_INT_ENA	I2C_TXFIFO_OVF_INT_ENA	I2C_NACK_INT_ENA	I2C_TRANS_START_INT_ENA	I2C_TIME_OUT_INT_ENA	I2C_TRANS_COMPLETE_INT_ENA	I2C_MST_TXFIFO_UDF_INT_ENA	I2C_ARBITRATION_LOST_INT_ENA	I2C_BYTE_TRANS_DONE_INT_ENA	I2C_END_DETECT_INT_ENA	I2C_RXFIFO_OVF_INT_ENA	I2C_TXFIFO_WM_INT_ENA	I2C_RXFIFO_WM_INT_ENA
Bit	31 … 19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reset	0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

I2C_RXFIFO_WM_INT_ENA Write 1 to enable I2C_RXFIFO_WM_INT interrupt. (R/W)

I2C_TXFIFO_WM_INT_ENA Write 1 to enable I2C_TXFIFO_WM_INT interrupt. (R/W)

I2C_RXFIFO_OVF_INT_ENA Write 1 to enable I2C_RXFIFO_OVF_INT interrupt. (R/W)

I2C_END_DETECT_INT_ENA Write 1 to enable the I2C_END_DETECT_INT interrupt. (R/W)

I2C_BYTE_TRANS_DONE_INT_ENA Write 1 to enable the I2C_END_DETECT_INT interrupt. (R/W)

I2C_ARBITRATION_LOST_INT_ENA Write 1 to enable the I2C_ARBITRATION_LOST_INT interrupt. (R/W)

I2C_MST_TXFIFO_UDF_INT_ENA Write 1 to enable I2C_TRANS_COMPLETE_INT interrupt. (R/W)

I2C_TRANS_COMPLETE_INT_ENA Write 1 to enable the I2C_TRANS_COMPLETE_INT interrupt. (R/W)

I2C_TIME_OUT_INT_ENA Write 1 to enable the I2C_TIME_OUT_INT interrupt. (R/W)

I2C_TRANS_START_INT_ENA Write 1 to enable the I2C_TRANS_START_INT interrupt. (R/W)

I2C_NACK_INT_ENA Write 1 to enable I2C_SLAVE_STRETCH_INT interrupt. (R/W)

I2C_TXFIFO_OVF_INT_ENA Write 1 to enable I2C_TXFIFO_OVF_INT interrupt. (R/W)

I2C_RXFIFO_UDF_INT_ENA Write 1 to enable I2C_RXFIFO_UDF_INT interrupt. (R/W)

I2C_SCL_ST_TO_INT_ENA Write 1 to enable I2C_SCL_ST_TO_INT interrupt. (R/W)

I2C_SCL_MAIN_ST_TO_INT_ENA Write 1 to enable I2C_SCL_MAIN_ST_TO_INT interrupt. (R/W)

I2C_DET_START_INT_ENA Write 1 to enable I2C_DET_START_INT interrupt. (R/W)

I2C_SLAVE_STRETCH_INT_ENA Write 1 to enable I2C_SLAVE_STRETCH_INT interrupt. (R/W)

I2C_GENERAL_CALL_INT_ENA Write 1 to enable I2C_GENARAL_CALL_INT interrupt. (R/W)

I2C_SLAVE_ADDR_UNMATCH_INT_ENA Write 1 to enable I2C_SLAVE_ADDR_UNMATCH_INT interrupt. (R/W)

Register 44.24. I2C_INT_STATUS_REG (0x002C)

Bits	Field	Reset
31–19	(reserved)	0 0 0 0 0 0 0 0 0 0 0 0 0
18	I2C_SLAVE_ADDR_UNMATCH_INT_ST	0
17	I2C_GENERAL_CALL_INT_ST	0
16	I2C_SLAVE_STRETCH_INT_ST	0
15	I2C_DET_START_INT_ST	0
14	I2C_SCL_MAIN_ST_TO_INT_ST	0
13	I2C_SCL_ST_TO_INT_ST	0
12	I2C_RXFIFO_UDF_INT_ST	0
11	I2C_TXFIFO_OVF_INT_ST	0
10	I2C_NACK_INT_ST	0
9	I2C_TRANS_START_INT_ST	0
8	I2C_TIME_OUT_INT_ST	0
7	I2C_TRANS_COMPLETE_INT_ST	0
6	I2C_MST_TXFIFO_UDF_INT_ST	0
5	I2C_ARBITRATION_LOST_INT_ST	0
4	I2C_BYTE_TRANS_DONE_INT_ST	0
3	I2C_END_DETECT_INT_ST	0
2	I2C_RXFIFO_OVF_INT_ST	0
1	I2C_TXFIFO_WM_INT_ST	0
0	I2C_RXFIFO_WM_INT_ST	0

I2C_RXFIFO_WM_INT_ST The masked interrupt status of I2C_RXFIFO_WM_INT interrupt. (RO)

I2C_TXFIFO_WM_INT_ST The masked interrupt status of I2C_TXFIFO_WM_INT interrupt. (RO)

I2C_RXFIFO_OVF_INT_ST The masked interrupt status of I2C_RXFIFO_OVF_INT interrupt. (RO)

I2C_END_DETECT_INT_ST The masked interrupt status of the I2C_END_DETECT_INT interrupt. (RO)

I2C_BYTE_TRANS_DONE_INT_ST The masked interrupt status of the I2C_END_DETECT_INT interrupt. (RO)

I2C_ARBITRATION_LOST_INT_ST The masked interrupt status of the I2C_ARBITRATION_LOST_INT interrupt. (RO)

I2C_MST_TXFIFO_UDF_INT_ST The masked interrupt status of I2C_TRANS_COMPLETE_INT interrupt. (RO)

I2C_TRANS_COMPLETE_INT_ST The masked interrupt status of the I2C_TRANS_COMPLETE_INT interrupt. (RO)

I2C_TIME_OUT_INT_ST The masked interrupt status of the I2C_TIME_OUT_INT interrupt. (RO)

I2C_TRANS_START_INT_ST The masked interrupt status of the I2C_TRANS_START_INT interrupt. (RO)

I2C_NACK_INT_ST The masked interrupt status of I2C_SLAVE_STRETCH_INT interrupt. (RO)

I2C_TXFIFO_OVF_INT_ST The masked interrupt status of I2C_TXFIFO_OVF_INT interrupt. (RO)

Continued on the next page...

Register 44.24. I2C_INT_STATUS_REG (0x002C)

Continued from the previous page...

I2C_RXFIFO_UDF_INT_ST The masked interrupt status of I2C_RXFIFO_UDF_INT interrupt. (RO)

I2C_SCL_ST_TO_INT_ST The masked interrupt status of I2C_SCL_ST_TO_INT interrupt. (RO)

I2C_SCL_MAIN_ST_TO_INT_ST The masked interrupt status of I2C_SCL_MAIN_ST_TO_INT interrupt. (RO)

I2C_DET_START_INT_ST The masked interrupt status of I2C_DET_START_INT interrupt. (RO)

I2C_SLAVE_STRETCH_INT_ST The masked interrupt status of I2C_SLAVE_STRETCH_INT interrupt. (RO)

I2C_GENERAL_CALL_INT_ST The masked interrupt status of I2C_GENARAL_CALL_INT interrupt. (RO)

I2C_SLAVE_ADDR_UNMATCH_INT_ST The masked interrupt status of I2C_SLAVE_ADDR_UNMATCH_INT interrupt. (RO)

Register 44.25. I2C_COMD0_REG (0x0058)

I2C_COMMAND0_DONE	(reserved)	I2C_COMMAND0	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_COMMAND0 Configures command 0.

It consists of three parts:

op_code is the command

1: WRITE

2: STOP

3: READ

4: END

6: RSTART

Byte_num represents the number of bytes that need to be sent or received.

ack_check_en, ack_exp, and ack are used to control the ACK bit. See I2C cmd structure 44.4-2 for more information.

(R/W)

I2C_COMMAND0_DONE Represents whether command 0 is done in I2C Master mode.

0: Not done

1: Done

(R/W/SS)

Register 44.26. I2C_COMD1_REG (0x005C)

I2C_COMMAND1_DONE	(reserved)	I2C_COMMAND1	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_COMMAND1 Configures command 1.

See details in I2C_CMD0_REG[13:0]. (R/W)

I2C_COMMAND1_DONE Represents whether command 1 is done in I2C Master mode.

0: Not done

1: Done

(R/W/SS)

Register 44.27. I2C_COMD2_REG (0x0060)

I2C_COMMAND2_DONE	(reserved)	I2C_COMMAND2	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_COMMAND2 Configures command 2. See details in I2C_CMD0_REG[13:0]. (R/W)

I2C_COMMAND2_DONE Represents whether command 2 is done in I2C Master mode.
0: Not done
1: Done
(R/W/SS)

Register 44.28. I2C_COMD3_REG (0x0064)

I2C_COMMAND3_DONE	(reserved)	I2C_COMMAND3	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_COMMAND3 Configures command 3. See details in I2C_CMD0_REG[13:0]. (R/W)

I2C_COMMAND3_DONE Represents whether command 3 is done in I2C Master mode.
0: Not done
1: Done
(R/W/SS)

Register 44.29. I2C_COMD4_REG (0x0068)

I2C_COMMAND4_DONE	(reserved)	I2C_COMMAND4	
31	30 ... 14	13 ... 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_COMMAND4 Configures command 4. See details in I2C_CMD0_REG[13:0]. (R/W)

I2C_COMMAND4_DONE Represents whether command 4 is done in I2C Master mode.

0: Not done

1: Done

(R/W/SS)

Register 44.30. I2C_COMD5_REG (0x006C)

I2C_COMMAND5_DONE	(reserved)	I2C_COMMAND5	
31	30 ... 14	13 ... 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_COMMAND5 Configures command 5. See details in I2C_CMD0_REG[13:0]. (R/W)

I2C_COMMAND5_DONE Represents whether command 5 is done in I2C Master mode.

0: Not done

1: Done

(R/W/SS)

Register 44.31. I2C_COMD6_REG (0x0070)

I2C_COMMAND6_DONE	(reserved)	I2C_COMMAND6	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_COMMAND6 Configures command 6. See details in I2C_CMD0_REG[13:0]. (R/W)

I2C_COMMAND6_DONE Represents whether command 6 is done in I2C Master mode.
0: Not done
1: Done
(R/W/SS)

Register 44.32. I2C_COMD7_REG (0x0074)

I2C_COMMAND7_DONE	(reserved)	I2C_COMMAND7	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

I2C_COMMAND7 Configures command 7. See details in I2C_CMD0_REG[13:0]. (R/W)

I2C_COMMAND7_DONE Represents whether command 7 is done in I2C Master mode.
0: Not done
1: Done
(R/W/SS)

Register 44.33. I2C_DATE_REG (0x00F8)

I2C_DATE	
31 … 0	
0x2201172	Reset

I2C_DATE Version control register. (R/W)

Register 44.34. I2C_TXFIFO_START_ADDR_REG (0x0100)

I2C_TXFIFO_START_ADDR	
31 … 0	
0	Reset

I2C_TXFIFO_START_ADDR Represents the I2C TX FIFO first address. (HRO)

Register 44.35. I2C_RXFIFO_START_ADDR_REG (0x0180)

I2C_RXFIFO_START_ADDR	
31 … 0	
0	Reset

I2C_RXFIFO_START_ADDR Represents the I2C RX FIFO first address. (HRO)

44.9.2 LP_I2C Registers

Register 44.36. LP_I2C_SCL_LOW_PERIOD_REG (0x0000)

(reserved)	LP_I2C_SCL_LOW_PERIOD	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_SCL_LOW_PERIOD Configures the low level width of the SCL Clock in master mode.

Measurement unit: i2c_sclk

(R/W)

Register 44.37. LP_I2C_SDA_HOLD_REG (0x0030)

(reserved)	LP_I2C_SDA_HOLD_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_SDA_HOLD_TIME Configures the time to hold the data after the falling edge of SCL.
Measurement unit: i2c_sclk
(R/W)

Register 44.38. LP_I2C_SDA_SAMPLE_REG (0x0034)

(reserved)	LP_I2C_SDA_SAMPLE_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_SDA_SAMPLE_TIME Configures the time for sampling SDA.
Measurement unit: i2c_sclk
(R/W)

Register 44.39. LP_I2C_SCL_HIGH_PERIOD_REG (0x0038)

(reserved)	LP_I2C_SCL_WAIT_HIGH_PERIOD	LP_I2C_SCL_HIGH_PERIOD	
31 … 16	15 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	Reset

LP_I2C_SCL_HIGH_PERIOD Configures for how long SCL remains high in master mode.
Measurement unit: i2c_sclk
(R/W)

LP_I2C_SCL_WAIT_HIGH_PERIOD Configures the SCL_FSM's waiting period for SCL high level in master mode.
Measurement unit: i2c_sclk
(R/W)

Register 44.40. LP_I2C_SCL_START_HOLD_REG (0x0040)

(reserved)	LP_I2C_SCL_START_HOLD_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	8	Reset

LP_I2C_SCL_START_HOLD_TIME Configures the time between the falling edge of SDA and the falling edge of SCL for a START condition.
Measurement unit: i2c_sclk
(R/W)

Register 44.41. LP_I2C_SCL_RSTART_SETUP_REG (0x0044)

(reserved)	LP_I2C_SCL_RSTART_SETUP_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	8	Reset

LP_I2C_SCL_RSTART_SETUP_TIME Configures the time between the positive edge of SCL and the negative edge of SDA for a RESTART condition.
Measurement unit: i2c_sclk
(R/W)

Register 44.42. LP_I2C_SCL_STOP_HOLD_REG (0x0048)

(reserved)	LP_I2C_SCL_STOP_HOLD_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	8	Reset

LP_I2C_SCL_STOP_HOLD_TIME Configures the delay after the STOP condition.
Measurement unit: i2c_sclk
(R/W)

Register 44.43. LP_I2C_SCL_STOP_SETUP_REG (0x004C)

(reserved)	LP_I2C_SCL_STOP_SETUP_TIME	
31 … 9	8 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	8	Reset

LP_I2C_SCL_STOP_SETUP_TIME Configures the time between the rising edge of SCL and the rising edge of SDA.
Measurement unit: i2c_sclk
(R/W)

Register 44.44. LP_I2C_SCL_ST_TIME_OUT_REG (0x0078)

(reserved)	LP_I2C_SCL_ST_TO_I2C	
31 … 5	4 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0x10	Reset

LP_I2C_SCL_ST_TO_I2C Configures the threshold value of SCL_FSM state unchanged period. It should be no more than 23.
Measurement unit: i2c_sclk
(R/W)

Register 44.45. LP_I2C_SCL_MAIN_ST_TIME_OUT_REG (0x007C)

(reserved)	LP_I2C_SCL_MAIN_ST_TO_I2C	
31 … 5	4 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0x10	Reset

LP_I2C_SCL_MAIN_ST_TO_I2C Configures the threshold value of SCL_MAIN_FSM state unchanged period. It should be no more than 23.
Measurement unit: i2c_sclk
(R/W)

Register 44.46. LP_I2C_CTR_REG (0x0004)

Bits	Field	Reset
31–12	(reserved)	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
11	LP_I2C_CONF_UPGATE	0
10	LP_I2C_FSM_RST	0
9	LP_I2C_ARBITRATION_EN	1
8	LP_I2C_CLK_EN	0
7	LP_I2C_RX_LSB_FIRST	0
6	LP_I2C_TX_LSB_FIRST	0
5	LP_I2C_TRANS_START	0
4	(reserved)	0
3	LP_I2C_RX_FULL_ACK_LEVEL	1
2	LP_I2C_SAMPLE_SCL_LEVEL	0
1–0	(reserved)	0 0

LP_I2C_SAMPLE_SCL_LEVEL Configures the sample mode for SDA.
1: Sample SDA data on the SCL low level.
0: Sample SDA data on the SCL high level.
(R/W)

LP_I2C_RX_FULL_ACK_LEVEL Configures the ACK value that needs to be sent by master when the rx_fifo_cnt has reached the threshold. (R/W)

LP_I2C_TRANS_START Configures to start sending the data in txfifo for slave.
0: No effect
1: Start
(WT)

LP_I2C_TX_LSB_FIRST Configures to control the sending order for data to be sent.
1: send data from the least significant bit
0: send data from the most significant bit
(R/W)

LP_I2C_RX_LSB_FIRST Configures to control the storage order for received data.
1: receive data from the least significant bit
0: receive data from the most significant bit
(R/W)

LP_I2C_CLK_EN Configures whether to gate clock signal for registers.
0: Support clock only when registers are read or written to by software
1: Force clock on for registers.
(R/W)

LP_I2C_ARBITRATION_EN Configures to enable I2C bus arbitration detection.
0: No effect
1: Enable
(R/W)

LP_I2C_FSM_RST Configures to reset the SCL_FSM.
0: No effect
1: Reset
(WT)

LP_I2C_CONF_UPGATE Configures this bit for synchronization.
0: No effect
1: Synchronize
(WT)

Register 44.47. LP_I2C_TO_REG (0x000C)

(reserved)	LP_I2C_TIME_OUT_EN	LP_I2C_TIME_OUT_VALUE	
31 6	5	4 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0x10	Reset

LP_I2C_TIME_OUT_VALUE Configures the timeout threshold period for SCL stucking at high or low level. The actual period is 2^(reg_time_out_value +1)−1.
Measurement unit: i2c_sclk.
(R/W)

LP_I2C_TIME_OUT_EN Configures to enable time out control.
0: No effect
1: Enable
(R/W)

Register 44.48. LP_I2C_FIFO_CONF_REG (0x0018)

	(reserved)	LP_I2C_FIFO_PRT_EN	LP_I2C_TX_FIFO_RST	LP_I2C_RX_FIFO_RST	(reserved)	LP_I2C_NONFIFO_EN	(reserved)	LP_I2C_TXFIFO_WM_THRHD	(reserved)	LP_I2C_RXFIFO_WM_THRHD
Bits	31 … 15	14	13	12	11	10	9	8 … 5	4	3 … 0
Reset	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	1	0	0	0	0	0	0x2	0	0x6

LP_I2C_RXFIFO_WM_THRHD Configures the water mark threshold of RXFIFO in nonfifo access mode. When LP_I2C_FIFO_PRT_EN is 1 and rx FIFO counter is bigger than LP_I2C_RXFIFO_WM_THRHD[4:0], LP_I2C_RXFIFO_WM_INT_RAW bit will be valid. (R/W)

LP_I2C_TXFIFO_WM_THRHD Configures the water mark threshold of TXFIFO in nonfifo access mode. When LP_I2C_FIFO_PRT_EN is 1 and rx FIFO counter is bigger than LP_I2C_TXFIFO_WM_THRHD[4:0], LP_I2C_TXFIFO_WM_INT_RAW bit will be valid. (R/W)

LP_I2C_NONFIFO_EN Configures to enable APB nonfifo access. (R/W)

LP_I2C_RX_FIFO_RST Configures to reset RXFIFO.
0: No effect
1: Reset
(R/W)

LP_I2C_TX_FIFO_RST Configures to reset TXFIFO. 0: No effect
1: Reset
(R/W)

LP_I2C_FIFO_PRT_EN Configures to enable FIFO pointer in non-fifo access mode. This bit controls the valid bits and the TX/RX FIFO overflow, underflow, full and empty interrupts.
0: No effect
1: Enable
(R/W)

Register 44.49. LP_I2C_FILTER_CFG_REG (0x0050)

(reserved)	LP_I2C_SDA_FILTER_EN	LP_I2C_SCL_FILTER_EN	LP_I2C_SDA_FILTER_THRES	LP_I2C_SCL_FILTER_THRES	
31 … 10	9	8	7 … 4	3 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	1	1	0	0	Reset

LP_I2C_SCL_FILTER_THRES Configures the threshold pulse width to be filtered on SCL. When a pulse on the SCL input has smaller width than this value, the I2C controller will ignore that pulse.
Measurement unit: i2c_sclk
(R/W)

LP_I2C_SDA_FILTER_THRES Configures the threshold pulse width to be filtered on SDA. When a pulse on the SDA input has smaller width than this value, the I2C controller will ignore that pulse.
Measurement unit: i2c_sclk
(R/W)

LP_I2C_SCL_FILTER_EN Configures to enable the filter function for SCL.
0: No effect
1: Enable
(R/W)

LP_I2C_SDA_FILTER_EN Configures to enable the filter function for SDA.
0: No effect
1: Enable
(R/W)

Register 44.50. LP_I2C_SCL_SP_CONF_REG (0x0080)

(reserved)	reserved	reserved	LP_I2C_SCL_RST_SLV_NUM	LP_I2C_SCL_RST_SLV_EN	
31 … 8	7	6	5 … 1	0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0	Reset

LP_I2C_SCL_RST_SLV_EN Configures to send out SCL pulses when I2C master is IDLE. The number of pulses equals to LP_I2C_SCL_RST_SLV_NUM[4:0]. (R/W/SC)

LP_I2C_SCL_RST_SLV_NUM Configure the pulses of SCL generated in I2C master mode.
Valid when LP_I2C_SCL_RST_SLV_EN is 1.
Measurement unit: i2c_sclk
(R/W)

Register 44.51. LP_I2C_SR_REG (0x0008)

Bits	31	30:28	27	26:24	23	22:18	17:13	12:8	7:5	4	3	2:1	0	
Field	(reserved)	LP_I2C_SCL_STATE_LAST	(reserved)	LP_I2C_SCL_MAIN_STATE_LAST	(reserved)	LP_I2C_TXFIFO_CNT	(reserved)	LP_I2C_RXFIFO_CNT	(reserved)	LP_I2C_BUS_BUSY	LP_I2C_ARB_LOST	(reserved)	LP_I2C_RESP_REC	
Value	0	0	0	0	0	0	0 0 0 0 0	0	0 0 0	0	0	0 0	0	Reset

LP_I2C_RESP_REC Represents the received ACK value in master mode or slave mode.
- 0: ACK
- 1: NACK
- (RO)

LP_I2C_ARB_LOST Represents whether the I2C controller loses control of SCL line.
- 0: No arbitration lost
- 1: Arbitration lost
- (RO)

LP_I2C_BUS_BUSY Represents the I2C bus state.
- 1: The I2C bus is busy transferring data
- 0: The I2C bus is in idle state
- (RO)

LP_I2C_RXFIFO_CNT Represents the number of data bytes received in RAM. (RO)

LP_I2C_TXFIFO_CNT Represents the number of data bytes to be sent. (RO)

LP_I2C_SCL_MAIN_STATE_LAST Represents the states of the I2C module state machine.
- 0: Idle
- 1: Address shift
- 2: ACK address
- 3: Rx data
- 4: Tx data
- 5: Send ACK
- 6: Wait ACK
- (RO)

LP_I2C_SCL_STATE_LAST Represents the states of the state machine used to produce SCL.
- 0: Idle
- 1: Start
- 2: Negative edge
- 3: Low
- 4: Positive edge
- 5: High
- 6: Stop
- (RO)

Register 44.52. LP_I2C_FIFO_ST_REG (0x0014)

(reserved)	LP_I2C_TXFIFO_WADDR	(reserved)	LP_I2C_TXFIFO_RADDR	(reserved)	LP_I2C_RXFIFO_WADDR	(reserved)	LP_I2C_RXFIFO_RADDR	
31 … 19	18 … 15	14	13 … 10	9	8 … 5	4	3 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0	0	0	0	Reset

LP_I2C_RXFIFO_RADDR Represents the offset address of the APB reading from RXFIFO (RO)

LP_I2C_RXFIFO_WADDR Represents the offset address of i2c module receiving data and writing to RXFIFO. (RO)

LP_I2C_TXFIFO_RADDR Represents the offset address of i2c module reading from TXFIFO. (RO)

LP_I2C_TXFIFO_WADDR Represents the offset address of APB bus writing to TXFIFO. (RO)

Register 44.53. LP_I2C_DATA_REG (0x001C)

(reserved)	LP_I2C_FIFO_RDATA	
31 … 8	7 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_FIFO_RDATA Represents the value of RXFIFO read data. (RO)

Register 44.54. LP_I2C_INT_RAW_REG (0x0020)

Bits	Field	Reset
31:16	(reserved)	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
15	LP_I2C_DET_START_INT_RAW	0
14	LP_I2C_SCL_MAIN_ST_TO_INT_RAW	0
13	LP_I2C_SCL_ST_TO_INT_RAW	0
12	LP_I2C_RXFIFO_UDF_INT_RAW	0
11	LP_I2C_TXFIFO_OVF_INT_RAW	0
10	LP_I2C_NACK_INT_RAW	0
9	LP_I2C_TRANS_START_INT_RAW	0
8	LP_I2C_TIME_OUT_INT_RAW	0
7	LP_I2C_TRANS_COMPLETE_INT_RAW	0
6	LP_I2C_MST_TXFIFO_UDF_INT_RAW	0
5	LP_I2C_ARBITRATION_LOST_INT_RAW	0
4	LP_I2C_BYTE_TRANS_DONE_INT_RAW	0
3	LP_I2C_END_DETECT_INT_RAW	0
2	LP_I2C_RXFIFO_OVF_INT_RAW	0
1	LP_I2C_TXFIFO_WM_INT_RAW	1
0	LP_I2C_RXFIFO_WM_INT_RAW	0

LP_I2C_RXFIFO_WM_INT_RAW The raw interrupt status of LP_I2C_RXFIFO_WM_INT interrupt. (R/SS/WTC)

LP_I2C_TXFIFO_WM_INT_RAW The raw interrupt status of LP_I2C_TXFIFO_WM_INT interrupt. (R/SS/WTC)

LP_I2C_RXFIFO_OVF_INT_RAW The raw interrupt status of LP_I2C_RXFIFO_OVF_INT interrupt. (R/SS/WTC)

LP_I2C_END_DETECT_INT_RAW The raw interrupt status of the LP_I2C_END_DETECT_INT interrupt. (R/SS/WTC)

LP_I2C_BYTE_TRANS_DONE_INT_RAW The raw interrupt status of the LP_I2C_END_DETECT_INT interrupt. (R/SS/WTC)

LP_I2C_ARBITRATION_LOST_INT_RAW The raw interrupt status of the LP_I2C_ARBITRATION_LOST_INT interrupt. (R/SS/WTC)

LP_I2C_MST_TXFIFO_UDF_INT_RAW The raw interrupt status of LP_I2C_TRANS_COMPLETE_INT interrupt. (R/SS/WTC)

LP_I2C_TRANS_COMPLETE_INT_RAW The raw interrupt status of the LP_I2C_TRANS_COMPLETE_INT interrupt. (R/SS/WTC)

LP_I2C_TIME_OUT_INT_RAW The raw interrupt status of the LP_I2C_TIME_OUT_INT interrupt. (R/SS/WTC)

Continued on the next page...

Register 44.54. LP_I2C_INT_RAW_REG (0x0020)

Continued from the previous page...

LP_I2C_TRANS_START_INT_RAW The raw interrupt status of the LP_I2C_TRANS_START_INT interrupt. (R/SS/WTC)

LP_I2C_NACK_INT_RAW The raw interrupt status of LP_I2C_SLAVE_STRETCH_INT interrupt. (R/SS/WTC)

LP_I2C_TXFIFO_OVF_INT_RAW The raw interrupt status of LP_I2C_TXFIFO_OVF_INT interrupt. (R/SS/WTC)

LP_I2C_RXFIFO_UDF_INT_RAW The raw interrupt status of LP_I2C_RXFIFO_UDF_INT interrupt. (R/SS/WTC)

LP_I2C_SCL_ST_TO_INT_RAW The raw interrupt status of LP_I2C_SCL_ST_TO_INT interrupt. (R/SS/WTC)

LP_I2C_SCL_MAIN_ST_TO_INT_RAW The raw interrupt status of LP_I2C_SCL_MAIN_ST_TO_INT interrupt. (R/SS/WTC)

LP_I2C_DET_START_INT_RAW The raw interrupt status of LP_I2C_DET_START_INT interrupt. (R/SS/WTC)

Register 44.55. LP_I2C_INT_CLR_REG (0x0024)

Bits	Field	Reset
31–16	(reserved)	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
15	LP_I2C_DET_START_INT_CLR	0
14	LP_I2C_SCL_MAIN_ST_TO_INT_CLR	0
13	LP_I2C_SCL_ST_TO_INT_CLR	0
12	LP_I2C_RXFIFO_UDF_INT_CLR	0
11	LP_I2C_TXFIFO_OVF_INT_CLR	0
10	LP_I2C_NACK_INT_CLR	0
9	LP_I2C_TRANS_START_INT_CLR	0
8	LP_I2C_TIME_OUT_INT_CLR	0
7	LP_I2C_TRANS_COMPLETE_INT_CLR	0
6	LP_I2C_MST_TXFIFO_UDF_INT_CLR	0
5	LP_I2C_ARBITRATION_LOST_INT_CLR	0
4	LP_I2C_BYTE_TRANS_DONE_INT_CLR	0
3	LP_I2C_END_DETECT_INT_CLR	0
2	LP_I2C_RXFIFO_OVF_INT_CLR	0
1	LP_I2C_TXFIFO_WM_INT_CLR	0
0	LP_I2C_RXFIFO_WM_INT_CLR	0

LP_I2C_RXFIFO_WM_INT_CLR Write 1 to clear LP_I2C_RXFIFO_WM_INT interrupt. (WT)

LP_I2C_TXFIFO_WM_INT_CLR Write 1 to clear LP_I2C_TXFIFO_WM_INT interrupt. (WT)

LP_I2C_RXFIFO_OVF_INT_CLR Write 1 to clear LP_I2C_RXFIFO_OVF_INT interrupt. (WT)

LP_I2C_END_DETECT_INT_CLR Write 1 to clear the LP_I2C_END_DETECT_INT interrupt. (WT)

LP_I2C_BYTE_TRANS_DONE_INT_CLR Write 1 to clear the LP_I2C_END_DETECT_INT interrupt. (WT)

LP_I2C_ARBITRATION_LOST_INT_CLR Write 1 to clear the LP_I2C_ARBITRATION_LOST_INT interrupt. (WT)

LP_I2C_MST_TXFIFO_UDF_INT_CLR Write 1 to clear LP_I2C_TRANS_COMPLETE_INT interrupt. (WT)

LP_I2C_TRANS_COMPLETE_INT_CLR Write 1 to clear the LP_I2C_TRANS_COMPLETE_INT interrupt. (WT)

LP_I2C_TIME_OUT_INT_CLR Write 1 to clear the LP_I2C_TIME_OUT_INT interrupt. (WT)

LP_I2C_TRANS_START_INT_CLR Write 1 to clear the LP_I2C_TRANS_START_INT interrupt. (WT)

LP_I2C_NACK_INT_CLR Write 1 to clear LP_I2C_SLAVE_STRETCH_INT interrupt. (WT)

LP_I2C_TXFIFO_OVF_INT_CLR Write 1 to clear LP_I2C_TXFIFO_OVF_INT interrupt. (WT)

LP_I2C_RXFIFO_UDF_INT_CLR Write 1 to clear LP_I2C_RXFIFO_UDF_INT interrupt. (WT)

LP_I2C_SCL_ST_TO_INT_CLR Write 1 to clear LP_I2C_SCL_ST_TO_INT interrupt. (WT)

LP_I2C_SCL_MAIN_ST_TO_INT_CLR Write 1 to clear LP_I2C_SCL_MAIN_ST_TO_INT interrupt. (WT)

LP_I2C_DET_START_INT_CLR Write 1 to clear LP_I2C_DET_START_INT interrupt. (WT)

Register 44.56. LP_I2C_INT_ENA_REG (0x0028)

31–16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
(reserved)	LP_I2C_DET_START_INT_ENA	LP_I2C_SCL_MAIN_ST_TO_INT_ENA	LP_I2C_SCL_ST_TO_INT_ENA	LP_I2C_RXFIFO_UDF_INT_ENA	LP_I2C_TXFIFO_OVF_INT_ENA	LP_I2C_NACK_INT_ENA	LP_I2C_TRANS_START_INT_ENA	LP_I2C_TIME_OUT_INT_ENA	LP_I2C_TRANS_COMPLETE_INT_ENA	LP_I2C_MST_TXFIFO_UDF_INT_ENA	LP_I2C_ARBITRATION_LOST_INT_ENA	LP_I2C_BYTE_TRANS_DONE_INT_ENA	LP_I2C_END_DETECT_INT_ENA	LP_I2C_RXFIFO_OVF_INT_ENA	LP_I2C_TXFIFO_WM_INT_ENA	LP_I2C_RXFIFO_WM_INT_ENA	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

LP_I2C_RXFIFO_WM_INT_ENA Write 1 to enable LP_I2C_RXFIFO_WM_INT interrupt. (R/W)

LP_I2C_TXFIFO_WM_INT_ENA Write 1 to enable LP_I2C_TXFIFO_WM_INT interrupt. (R/W)

LP_I2C_RXFIFO_OVF_INT_ENA Write 1 to enable LP_I2C_RXFIFO_OVF_INT interrupt. (R/W)

LP_I2C_END_DETECT_INT_ENA Write 1 to enable the LP_I2C_END_DETECT_INT interrupt. (R/W)

LP_I2C_BYTE_TRANS_DONE_INT_ENA Write 1 to enable the LP_I2C_END_DETECT_INT interrupt. (R/W)

LP_I2C_ARBITRATION_LOST_INT_ENA Write 1 to enable the LP_I2C_ARBITRATION_LOST_INT interrupt. (R/W)

LP_I2C_MST_TXFIFO_UDF_INT_ENA Write 1 to enable LP_I2C_TRANS_COMPLETE_INT interrupt. (R/W)

LP_I2C_TRANS_COMPLETE_INT_ENA Write 1 to enable the LP_I2C_TRANS_COMPLETE_INT interrupt. (R/W)

LP_I2C_TIME_OUT_INT_ENA Write 1 to enable the LP_I2C_TIME_OUT_INT interrupt. (R/W)

LP_I2C_TRANS_START_INT_ENA Write 1 to enable the LP_I2C_TRANS_START_INT interrupt. (R/W)

LP_I2C_NACK_INT_ENA Write 1 to enable LP_I2C_SLAVE_STRETCH_INT interrupt. (R/W)

LP_I2C_TXFIFO_OVF_INT_ENA Write 1 to enable LP_I2C_TXFIFO_OVF_INT interrupt. (R/W)

LP_I2C_RXFIFO_UDF_INT_ENA Write 1 to enable LP_I2C_RXFIFO_UDF_INT interrupt. (R/W)

LP_I2C_SCL_ST_TO_INT_ENA Write 1 to enable LP_I2C_SCL_ST_TO_INT interrupt. (R/W)

LP_I2C_SCL_MAIN_ST_TO_INT_ENA Write 1 to enable LP_I2C_SCL_MAIN_ST_TO_INT interrupt. (R/W)

LP_I2C_DET_START_INT_ENA Write 1 to enable LP_I2C_DET_START_INT interrupt. (R/W)

Register 44.57. LP_I2C_INT_STATUS_REG (0x002C)

Bits	31–16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	(reserved)	LP_I2C_DET_START_INT_ST	LP_I2C_SCL_MAIN_ST_TO_INT_ST	LP_I2C_SCL_ST_TO_INT_ST	LP_I2C_RXFIFO_UDF_INT_ST	LP_I2C_TXFIFO_OVF_INT_ST	LP_I2C_NACK_INT_ST	LP_I2C_TRANS_START_INT_ST	LP_I2C_TIME_OUT_INT_ST	LP_I2C_TRANS_COMPLETE_INT_ST	LP_I2C_MST_TXFIFO_UDF_INT_ST	LP_I2C_ARBITRATION_LOST_INT_ST	LP_I2C_BYTE_TRANS_DONE_INT_ST	LP_I2C_END_DETECT_INT_ST	LP_I2C_RXFIFO_OVF_INT_ST	LP_I2C_TXFIFO_WM_INT_ST	LP_I2C_RXFIFO_WM_INT_ST
Reset	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

LP_I2C_RXFIFO_WM_INT_ST The masked interrupt status of LP_I2C_RXFIFO_WM_INT interrupt. (RO)

LP_I2C_TXFIFO_WM_INT_ST The masked interrupt status of LP_I2C_TXFIFO_WM_INT interrupt. (RO)

LP_I2C_RXFIFO_OVF_INT_ST The masked interrupt status of LP_I2C_RXFIFO_OVF_INT interrupt. (RO)

LP_I2C_END_DETECT_INT_ST The masked interrupt status of the LP_I2C_END_DETECT_INT interrupt. (RO)

LP_I2C_BYTE_TRANS_DONE_INT_ST The masked interrupt status of the LP_I2C_END_DETECT_INT interrupt. (RO)

LP_I2C_ARBITRATION_LOST_INT_ST The masked interrupt status of the LP_I2C_ARBITRATION_LOST_INT interrupt. (RO)

LP_I2C_MST_TXFIFO_UDF_INT_ST The masked interrupt status of LP_I2C_TRANS_COMPLETE_INT interrupt. (RO)

LP_I2C_TRANS_COMPLETE_INT_ST The masked interrupt status of the LP_I2C_TRANS_COMPLETE_INT interrupt. (RO)

LP_I2C_TIME_OUT_INT_ST The masked interrupt status of the LP_I2C_TIME_OUT_INT interrupt. (RO)

LP_I2C_TRANS_START_INT_ST The masked interrupt status of the LP_I2C_TRANS_START_INT interrupt. (RO)

Continued on the next page...

Register 44.57. LP_I2C_INT_STATUS_REG (0x002C)

Continued from the previous page...

LP_I2C_NACK_INT_ST The masked interrupt status of LP_I2C_SLAVE_STRETCH_INT interrupt. (RO)

LP_I2C_TXFIFO_OVF_INT_ST The masked interrupt status of LP_I2C_TXFIFO_OVF_INT interrupt. (RO)

LP_I2C_RXFIFO_UDF_INT_ST The masked interrupt status of LP_I2C_RXFIFO_UDF_INT interrupt. (RO)

LP_I2C_SCL_ST_TO_INT_ST The masked interrupt status of LP_I2C_SCL_ST_TO_INT interrupt. (RO)

LP_I2C_SCL_MAIN_ST_TO_INT_ST The masked interrupt status of LP_I2C_SCL_MAIN_ST_TO_INT interrupt. (RO)

LP_I2C_DET_START_INT_ST The masked interrupt status of LP_I2C_DET_START_INT interrupt. (RO)

Register 44.58. LP_I2C_COMD0_REG (0x0058)

LP_I2C_COMMAND0_DONE	(reserved)	LP_I2C_COMMAND0	
31	30 ... 14	13 ... 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_COMMAND0 Configures command 0.
It consists of three parts:
op_code is the command
1: WRITE
2: STOP
3: READ
4: END
6: RSTART
Byte_num represents the number of bytes that need to be sent or received.
ack_check_en, ack_exp and ack are used to control the ACK bit. See I2C cmd structure 44.4-2 for more information. (R/W)

LP_I2C_COMMAND0_DONE Represents whether command 0 is done in I2C Master mode.
0: Not done
1: Done
(R/W/SS)

Register 44.59. LP_I2C_COMD1_REG (0x005C)

LP_I2C_COMMAND1_DONE	(reserved)	LP_I2C_COMMAND1	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_COMMAND1 Configures command 1.
See details in I2C_CMD0_REG[13:0]. (R/W)

LP_I2C_COMMAND1_DONE Represents whether command 1 is done in I2C Master mode.
0: Not done
1: Done
(R/W/SS)

Register 44.60. LP_I2C_COMD2_REG (0x0060)

LP_I2C_COMMAND2_DONE	(reserved)	LP_I2C_COMMAND2	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_COMMAND2 Configures command 2. See details in I2C_CMD0_REG[13:0]. (R/W)

LP_I2C_COMMAND2_DONE Represents whether command 2 is done in I2C Master mode.
0: Not done
1: Done
(R/W/SS)

Register 44.61. LP_I2C_COMD3_REG (0x0064)

LP_I2C_COMMAND3_DONE	(reserved)	LP_I2C_COMMAND3	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_COMMAND3 Configures command 3. See details in I2C_CMD0_REG[13:0]. (R/W)

LP_I2C_COMMAND3_DONE Represents whether command 3 is done in I2C Master mode.
- 0: Not done
- 1: Done

(R/W/SS)

Register 44.62. LP_I2C_COMD4_REG (0x0068)

LP_I2C_COMMAND4_DONE	(reserved)	LP_I2C_COMMAND4	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_COMMAND4 Configures command 4. See details in I2C_CMD0_REG[13:0]. (R/W)

LP_I2C_COMMAND4_DONE Represents whether command 4 is done in I2C Master mode.
- 0: Not done
- 1: Done

(R/W/SS)

Register 44.63. LP_I2C_COMD5_REG (0x006C)

LP_I2C_COMMAND5_DONE	(reserved)	LP_I2C_COMMAND5	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_COMMAND5 Configures command 5. See details in I2C_CMD0_REG[13:0]. (R/W)

LP_I2C_COMMAND5_DONE Represents whether command 5 is done in I2C Master mode.

0: Not done

1: Done

(R/W/SS)

Register 44.64. LP_I2C_COMD6_REG (0x0070)

LP_I2C_COMMAND6_DONE	(reserved)	LP_I2C_COMMAND6	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_COMMAND6 Configures command 6. See details in I2C_CMD0_REG[13:0]. (R/W)

LP_I2C_COMMAND6_DONE Represents whether command 6 is done in I2C Master mode.

0: Not done

1: Done

(R/W/SS)

Register 44.65. LP_I2C_COMD7_REG (0x0074)

LP_I2C_COMMAND7_DONE	(reserved)	LP_I2C_COMMAND7	
31	30 … 14	13 … 0	
0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	Reset

LP_I2C_COMMAND7 Configures command 7. See details in I2C_CMD0_REG[13:0]. (R/W)

LP_I2C_COMMAND7_DONE Represents whether command 7 is done in I2C Master mode.

0: Not done

1: Done

(R/W/SS)

Register 44.66. LP_I2C_DATE_REG (0x00F8)

LP_I2C_DATE	
31 … 0	
0x2201143	Reset

LP_I2C_DATE Version control register. (R/W)

Register 44.67. LP_I2C_TXFIFO_START_ADDR_REG (0x0100)

LP_I2C_TXFIFO_START_ADDR	
31 … 0	
0	Reset

LP_I2C_TXFIFO_START_ADDR Represents the I2C txfifo first address. (HRO)

Register 44.68. LP_I2C_RXFIFO_START_ADDR_REG (0x0180)

LP_I2C_RXFIFO_START_ADDR	
31 ... 0	
0	Reset

LP_I2C_RXFIFO_START_ADDR Represents the I2C rxfifo first address. (HRO)